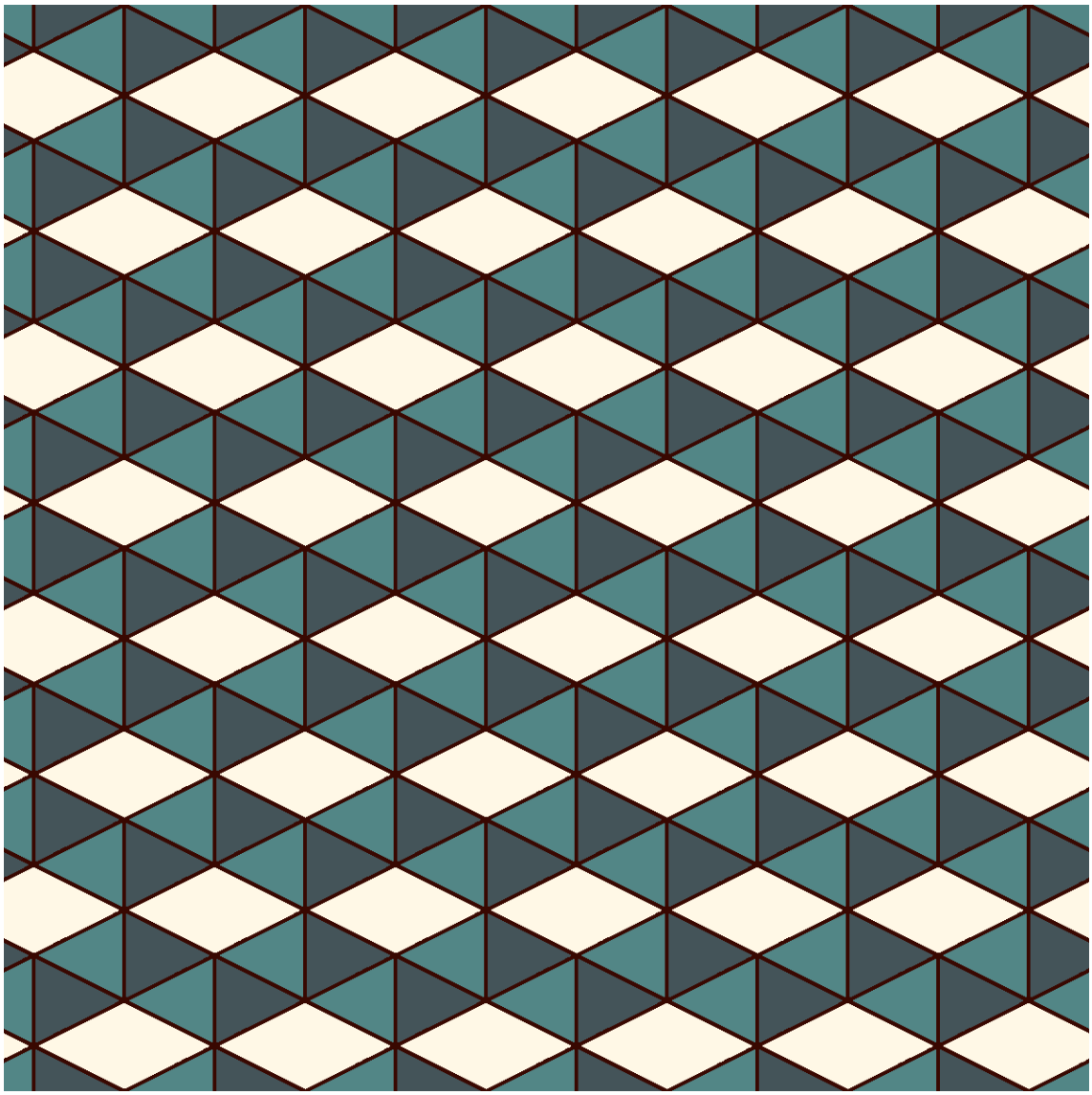
Higher order pattern unification using scope graphs

Master's Thesis



Tudor Andrei

Higher order pattern unification using scope graphs

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Tudor Andrei
born in Bucharest, Romania



Programming Languages Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

© 2026 Tudor Andrei.

Cover graphic by `funkyplayer`, Adobe Stock, used under license.

Higher order pattern unification using scope graphs

Author: Tudor Andrei
Student id: 5495822
Email: T.Andrei@student.tudelft.nl

Abstract

This thesis investigates whether higher order pattern unification can be integrated with a type checker that represents name binding using scope graphs. The work is carried out in Statix, a constraint language in the Spoofax language workbench. Statix uses scope graphs for declarative name resolution and provides first order unification as part of its constraint solver. However, inference in dependently typed languages can require higher order unification when the inferred solution depends on variables introduced by surrounding binders.

We implement a small dependently typed language in Statix and extend its type checker with higher order pattern unification. Scope graph information is used to determine which bound variables may occur in a solution and to construct well scoped higher order solutions. The implementation supports the pattern cases considered in this thesis, but does not support postponement of constraints that may become solvable later.

We also implement the core of the same language in Elpi using Higher Order Abstract Syntax. The two implementations are used to compare explicit scope based binding with binding provided by a higher order meta language. The comparison shows that Elpi provides direct support for lexical binding, substitution, and higher order unification, while scope graphs provide a direct representation of non lexical name resolution such as modules, qualified references, and forward references.

Thesis Committee:

Chair:	Dr. J.G.H. (Jesper) Cockx, Faculty EEMCS, TU Delft
Committee Member:	Dr. S. (Sebastijan) Dumančić, Faculty EEMCS, TU Delft
University Supervisor:	Dr. J.G.H. (Jesper) Cockx, Faculty EEMCS, TU Delft

Preface

Preface here.

Tudor Andrei
Delft, the Netherlands
October 7, 2026

Contents

Preface iii

Contents v

List of Figures vii

List of Tables ix

1 Introduction 1
- 1.1 Contributions . . . 2

2 Background 5
- 2.1 Dependent Types . . . 5
- 2.2 Name binding . . . 7
- 2.3 Name binding in dependent types . . . 7
- 2.4 Scope Graphs . . . 8
- 2.5 Another way to model name binding: Higher-Order Abstract Syntax . . . 10

3 Higher order unification 13
- 3.1 Motivation for having HOU in the language . . . 13
- 3.2 Cases in which type checking requires HOU . . . 13
- 3.3 Metavariable solutions are functions . . . 14
- 3.4 Additions to higher order unification and the pattern fragment . . . 15

4 A Language with Dependent Types in Statix 19
- 4.1 Language . . . 19
- 4.2 Variables and substitution . . . 20
- 4.3 Beta reduction . . . 23
- 4.4 Equality . . . 26
- 4.5 Statements and forward references . . . 27
- 4.6 Modules . . . 29

5 Inference and Unification 33
- 5.1 Typed holes . . . 33
- 5.2 Scoping of metavariables . . . 34
- 5.3 Metavariables in scope graphs . . . 34
- 5.4 Higher order pattern unification in Statix . . . 36
- 5.5 Discussion . . . 38

6 A comparison of scope graphs and HOAS **39**
6.1 Logic programming in Elpi 39
6.2 Defining the language in HOAS 41
6.3 HOAS as an alternative to scope graphs 44

7 Related work **47**
7.1 Higher order unification and dependent type elaboration 47
7.2 Scope graphs and dependent typing in Statix 48
7.3 Higher Order Abstract Syntax and logic programming 48

8 Conclusion **49**
8.1 Future Work 49
8.2 Reproducibility 50
8.3 AI usage 50

Bibliography **51**

Acronyms **55**

List of Figures

4.1 Scope graph of listing 4.2 21
4.2 Resulting scope graph of 4.6.3 32

5.1 Example scope graph with metavariables 35

List of Tables

Chapter 1

Introduction

Having a strong type system has become a defining characteristic of many modern programming languages. Beyond basic error detection, expressive type systems allow developers to encode invariants directly into the structure of their programs, shifting entire classes of bugs from runtime to compile time. Types serve as a precise form of documentation, making assumptions and guarantees explicit and enforceable.

Dependent type systems represent a significant step in this progression. They extend conventional type systems by allowing types to depend on values. This added expressiveness enables the encoding of precise specifications. For instance, types that capture the exact length of a list or enforce domain-specific constraints. Through the Curry-Howard correspondence [26], such type systems can express logical proofs. Writing a well-typed program becomes equivalent to constructing a proof of a logical proposition. As a result, languages with dependent types can function as proof assistants for propositional logic with quantifiers. Practical realizations of these ideas can be seen in systems such as Lean, Agda, and Idris 2, where programming and formal verification are tightly integrated. The dependent type calculus introduces Π-types of the form: $(x : A) \to B$, where x can appear in the definition of B, apart from the regular function type $A \to B$. x is only known when we apply the function on an argument. The actual value of the argument will then be substituted for x in B, to obtain the return type.

For a dependently typed language to be considered practical, it needs to perform as much type inference as possible to avoid requiring programmers to provide information that can be reconstructed from context. Consider the function that adds an element to the head of a length-indexed vector:

$$\mathit{cons} : (A : \mathit{Set}) \to (n : \mathit{Nat}) \to A \to \mathit{Vec}\ A\ n \to \mathit{Vec}\ A\ (\mathit{suc}\ n).$$

The higher-order nature of this problem follows from the binding structure of dependent types. In a dependent function type $(x : A) \to B$, the variable x is in scope in B, so the result of inference may itself depend on locally bound variables. Consequently, solving an equality between types can require inferring not only a term, but a function of the variables that are currently in scope which is more complex than first-order unification and motivates the use of higher-order unification [16, 17].

Binding introduced by dependent function types is, however, only one aspect of name binding in a programming language. Name resolution may also depend on modules, imports, inheritance, qualified names, and forward references. These visibility relations do not necessarily follow the lexical nesting of the abstract syntax tree [29, 21]. A language implementation for dependent types has to support both the binding structure that arises inside terms and types and the broader name-resolution mechanisms of the surrounding programming language.

Language workbenches provide a good setting in which to implement these concerns. They are environments designed to support the development of programming languages

[11] by providing reusable infrastructure for common compiler and tooling concerns. They exist primarily to reduce the engineering overhead associated with implementing languages from scratch, particularly for very common features such as parsing, name binding, type checking, and editor support. By elevating these concerns to declarative specifications, language workbenches allow language designers to focus on semantics rather than implementation details. Language workbenches enable rapid prototyping, facilitate experimentation with advanced type systems, and help correctness through well-defined meta-languages and tooling. As an example, in the case of dependent type languages, building all the necessary tooling from scratch is a difficult task. Type checking in such systems often involves combining mechanisms like higher-order unification, weak head normalization and careful handling of variable binding and scope. Language workbenches provide abstractions that address these challenges. Core concerns such as name resolution, binding structure, and unification are treated as first-class components, allowing language designers to focus on the semantics of their type systems rather than low-level implementation details.

Spoofax [30] is a language workbench developed at TU Delft. Static semantics in Spoofax can be specified using Statix [4], a constraint-based language for expressing typing and name-resolution rules. Statix represents name binding explicitly through scope graphs. Scope graphs model scopes as nodes and visibility relations as labeled edges, allowing lexical and non-lexical name-resolution mechanisms to be expressed within the same framework [4]. This makes them particularly suitable for features such as imports, qualified references, and forward references.

However, the term language and built-in unification mechanism of Statix are first-order. Binding structure is represented externally through the scope graph rather than through binders in the meta-language. As a result, operations needed by dependent type checking, such as substitution under binders and higher-order unification of metavariables, cannot simply be delegated to the host language and must instead be expressed explicitly.

A different approach is Higher-Order Abstract Syntax (HOAS) [24], where object language binders are represented using binders of the meta-language. In higher-order logic programming systems, this representation provides direct support for operations such as capture-avoiding substitution and higher-order unification. The trade-off is that such binding mechanisms are fundamentally lexical, whereas scope graphs are designed to represent more general visibility relations between scopes.

HOAS and Scope Graphs differ fundamentally in how they treat binding and unification. Since HOAS follows the syntax of simply typed lambda calculus, names are bound only by constructing a lambda abstraction while properties are asserted via implication. In return, users get capture-avoiding substitution, lexical scoping and higher order unification for free. While scope graphs provide strong support for modular and non-lexical scoping, they do not natively provide higher-order pattern unification, which is central to dependently typed systems.

This issue motivates the central question of this thesis: can higher-order pattern unification be integrated with a scope-graph-based constraint system in a principled and practical way (RQ1)? Furthermore, how does such an approach compare to higher-order logic programming systems when implementing a dependently typed language (RQ2)?

To address these questions, we design and evaluate a higher-order pattern unification algorithm tailored to a scope-graph-based setting, and analyze its implications for implementing dependently typed languages in Spoofax.

1.1 Contributions

This thesis addresses two research questions and makes the following contributions.

Higher-order pattern unification in scope-graph systems. We implement and study an algorithm for higher-order pattern unification in a constraint-based system that uses scope graphs for name binding. To validate the algorithm, we implement a minimal dependently typed core language in Statix, demonstrating that higher-order pattern unification can be integrated with scope-graph-based name resolution and supports core dependent type features.

Comparison of implementation paradigms for dependent typing. We provide a structured comparison between higher-order logic programming systems such as λ-Prolog and scope-graph-based constraint systems. The analysis qualitatively compares implementation trade-offs, including the treatment of binding, fresh name generation, modularity, and non-lexical scoping.

Chapter 2

Background

In this chapter we will argue about the importance of rigorous type systems, such as dependent type theory. We will also explain the unique binding requirements of this system and how it can be used to provide better editor services. Furthermore, we explain the theory behind two approaches to name binding: scope graphs and higher order abstract syntax.

A lot of effort has been made towards improving type safety of new languages. The benefits of strong type systems are easy to see: self-documenting code, more bugs prevented at compile time and stronger invariants. The latter is becoming increasingly important. If an identifier has a type, the programmer can make certain assumptions about its value. For example, in Rust, an invariant of reference values is that they can never be dangling. This property is guaranteed at compile time by the borrow checker. Furthermore, lifetime annotations in struct definitions such as:

```
struct A<'a> {
  pub s: &'a str
}
```

guarantee that the references to heap-allocated string values lives as long as the struct it belongs to, making it safe to access. Another invariant programmers might be familiar with are (non)-nullable values. Languages like TypeScript and Kotlin can perform type narrowing after the value is asserted as null or non-null. The type of a nullable value `x: Int?`, can be narrowed as so:

```
if x != null {
  // x guaranteed to be Int and not null here
}
```

The compiler's ability to check invariants strongly correlates with the safety of the code. However, in most conventional type systems, the set of invariants that can be checked is fixed by the compiler's implementation and users have no way to extend it.

2.1 Dependent Types

Dependent type systems, as introduced by Martin-Löf [15], can address this challenge. In a dependently typed language, types may depend on values, enabling programmers to express and verify arbitrary invariants within the type system itself. Moreover, the ability to bind arbitrary values within a type induces the need for type level computation (reduction). In the following sections we will rely on Agda [23] for our examples, since it is an easy to read language that implements Martin-Löf's theory of dependent types.

2.1.1 Function types and Pi types.

In the simply typed calculus, a function type $A \to B$ describes functions that take an argument of type A and return a result of type B. The return type B is fixed and cannot refer to the value of the argument. A *dependent function type* (or *Pi type*), written $\Pi\,(x : A).\, B(x)$ generalizes this with the return type $B(x)$ being able to mention the argument x. When the return type does not depend on the argument, the Pi type reduces to an ordinary function type $A \to B$. As a first example, consider a function `zeros` : $\Pi\,(x : \mathsf{Int}).\, \mathsf{Vec}\ \mathsf{Int}\ x$, which produces a vector of zeros with length x. This cannot be expressed at the type level with an ordinary function type in simply typed calculus.

In practice, many type arguments can be inferred from context. Most dependently typed languages allow such arguments to be marked as *implicit*, meaning they are automatically filled in by the type checker. The canonical example is the identity function, which can be given the type id : $\{A : \text{Type}\} \to A \to A$, where the braces indicate that A is implicit. The caller writes id 42 rather than id $\mathbb{N}$ 42, and the type checker infers that $A = \mathbb{N}$.

2.1.2 Data types and constructors.

Dependently typed languages support *indexed* data types, whose type signature carries information about the values it classifies. A data type may have *parameters*, which are fixed across all constructors, and *indices*, which may vary between constructors. This distinction is important: indices allow data types to be indexed by values, which is the key mechanism for encoding invariants in the type system.

It is often easier to think of types as propositions that say something about a class of values. If we can provide a value for a type, the type checker will assert the invariant described in the type. As an example, we can design an invariant which guarantees that a value is non-null. We will use the Option type to represent *null* as it is the idiomatic way to represent nullable values in pure languages.

Listing 2.1: Non-null invariant written in Agda

```
data Option (A : Set) : Set where
  null : Option A
  some : A → Option A

-- Non-null invariant expressed as a type over Option values
-- Only values constructed with some will make this proof true
data IsNonNull {A : Set} : Option A → Set where
  is-non-null : ∀ {x} → IsNonNull (some x)

-- the caller must provide a proof of non-nullity
unwrap : {A : Set} → (m : Option A) → IsNonNull m → A
unwrap (some x) is-non-null = x

is-non-null : IsNonNull (some 42) -- will compile
is-non-null : IsNonNull null -- won't compile because the type has
    0 inhabitants
```

In this example, the non-null invariant is required by the `unwrap` function. The type checker will accept the expression `unwrap x proof` in two cases: a) x is constructed with `some` at the call site or b) we receive the proof that x is not null from another function.

One of the most important feature of dependent type theory, that allows us to use values (or any terms for that matter) at compile time is the particular name binding structure. In

the previous example, `unwrap` can use the **value** of its first argument in the type signature, by binding it to the name m.

2.2 Name binding

This section will give an overview of name bindings in the context of static analysis. We will also discuss the relation between scoping and name binding.

In the literature, there are many types of name binding, but this work will focus on static name binding (binding that happens at compile time). In static analysis of programs it is common to infer information about a name when it is first introduced (*declared*), while later *references* of the identifier can re-use the collected information or refine it [29]. For well-formed program all references must be *bound* to a single declaration. However, in many programming languages used in practice, it is not always obvious how to map a reference to its declaration. This process is known as *name resolution* [29].

A common way to represent the result of name resolution is through *def-use chains*, which link each reference directly to its corresponding declaration [3]. Def-use analysis can be run once and its result can be reused by the compiler for different passes. Def-use chains are the backbone of type checking and numerous IDE services such as go-to-definition, find references, and rename refactoring.

The rules that govern name resolution are determined by the scoping discipline of the language. The most common discipline is *lexical scoping*, where the scope of a name is determined by its position in the AST. In a lexically scoped language, an inner scope may *shadow* a declaration from an enclosing scope, and resolution proceeds outward from the reference site until a matching declaration is found. Lexical scoping was popularized by Algol 60 and is used by the majority of modern programming languages [28]. However, many practical languages also feature *non-lexical* scoping mechanisms that cannot be explained solely by the AST hierarchy. Examples include module imports, class inheritance, qualified names, and forward-references. Non-lexical scoping increases the complexity of type checking. Since the scoping rules don't follow the hierarchy of the AST neither can the type checker. For instance, to type check forward references, the type checker cannot simply follow the AST, as it will encounter references of names that have not yet been declared. To account for this it should collect and process all available declarations within a scope before proceeding. It follows that scoping and name resolution are tightly coupled with the operational semantics of a type checker. In most languages, non-lexical scoping is handled in an ad-hoc manner embedded in the type checker and other static analysis. [21].

2.3 Name binding in dependent types

A dependently typed language adds subtle complexities to the name binding and resolution process. One difference to a simply typed language is the ability to bind values (of any type) in types. When performing an application $f\ x$, the value of x might be bound both by f's binder and by f's type. Another challenge of name binding comes with inference. In dependently typed languages inference is complex and requires higher-order unification as shown by [16, 19, 2]. This complexity arises from the presence of binders at the type level. In particular, when unifying two types, the solution can depend on these binders and will generally have the form of a function. More details about this process and how it is integrated in type checking, are given in Chapter 3.

The most common approach to add inference is to create "holes" (metavariables) in places where a term must be inferred. Here, term refers to either a value or a type, since in this type system we should infer both. A familiar example is Hindley-Milner type inference, where fresh type variables represent unknown monotypes and first-order unification

progressively determines their solutions. For every typeable expression, the Hindley-Milner inference algorithm computes a principal type scheme [18, 7]. Unlike ordinary bound variables, metavariables are initially unsolved and carry only the information required to ensure that they are well-scoped and well-typed, such as their type or kind and the context in which they were introduced. As elaboration proceeds, uses of expressions containing metavariables generate unification constraints. Solving these constraints assigns *solutions* to the metavariables, progressively refining the partial term until the missing information has been reconstructed [12, 10].

Language implementations view type checking and constraint solving as separate systems. The interface between the two components can look as follows:

1. The solver can create new metavariables (by itself or when requested)
2. Type checker can publish a constraint for unifying two terms
3. Type checker can give a term with "holes" and receive back a term with metavariables substituted for their current solutions (if they exist) from the solver

Point 2. is perhaps vague. Does the type checker expect a response from the solver or does it just move on? This question motivates the existence of another approach for unification called *dynamic pattern unification*. The dynamic approach distinguishes between 3 states for constraints: solvable, impossible to solve or delayed. The latter suggests that constraints which come later might help us solve constraints we accumulated earlier. Furthermore, unification becomes decoupled from the AST and requires a form of non-lexical scoping. We need to capture the required context (bindings, telescopes) for a constraint and store it.

As highlighted by point 3, the type checker needs to know information about a metavariable. We make a distinction between the scope in which a metavariable is created and the scope in which it is solved. If we strictly follow lexical scoping, the scope in which we query might not be a descendant of the scope in which we instantiate the metavariable. Therefore, metavariables and their solutions need to live in a global, mutable scope.

2.4 Scope Graphs

Modern programming languages combine multiple scoping mechanisms, including lexical scoping induced by syntactic structure and non-lexical scoping arising from constructs such as imports, inheritance, or other visibility relations. Traditional formulations of name resolution, for instance based on hierarchical symbol tables, are well-suited for purely lexical scoping but become increasingly ad hoc when extended to accommodate such non-lexical mechanisms. Scope graphs provide a uniform representation in which both forms of scoping are modeled within a single formalism, by representing scopes as nodes in a graph and scoping relations as labeled edges.

Formally, a scope graph is a directed labeled graph $G = (S, E, D)$, where S is a set of scopes, D is a set of declarations, and $E \subseteq S \times L \times S$ is a set of labeled edges between scopes, for some set of labels L. Each declaration $d \in D$ is associated with a scope in S and introduces a name. References to names occur in scopes and must be resolved to declarations. Lexical scoping is typically represented by edges corresponding to syntactic nesting, whereas non-lexical scoping mechanisms, such as imports or inheritance, are represented in the same graph as additional labeled edges [4]. This representation abstracts from the concrete language constructs and instead captures name binding in terms of reachability in the graph.

Name resolution is defined as a path search problem over the scope graph. Given a reference to a name x in a scope $s \in S$, resolution amounts to finding a declaration d of x in some scope s' such that there exists a path from s to s' that satisfies a given resolution policy. Such

a policy constrains the admissible sequences of edge labels and typically induces a preference ordering over paths in order to model shadowing. In particular, declarations that are reachable via preferred paths (for example, shorter paths or paths following specific edge labels) take precedence over others, ensuring that more local declarations shadow those that are more distant in the graph.

In *Statix*, scope graphs are integrated into a constraint-based framework in which both the construction of the graph and the resolution of names are expressed declaratively. Rather than constructing the scope graph procedurally, one specifies constraints that introduce scopes, declarations, and edges, as well as constraints that require certain references to resolve to declarations. Concretely, scope construction is expressed using constraints that generate fresh scopes, introduce declarations of names in scopes, and relate scopes via labeled edges. Name resolution is expressed by constraints of the form $\texttt{resolve}(s, x) = d$, requiring that the reference to x in scope s resolves to declaration d according to the resolution policy induced by the scope graph.

An advantage of Statix is that scope graph constraints interact with other semantic constraints, such as equality or typing constraints. For example, resolving a reference yields a declaration whose associated semantic information, such as a type, may be subject to further constraints. As a result, name resolution is not treated as a separate phase, but is instead integrated into a unified constraint system.

Constraint solving proceeds by incrementally refining a partial scope graph and a set of constraints. Resolution constraints are solved only when the relevant portion of the scope graph is sufficiently closed to guarantee a stable result (i.e. Statix knows we won't extend that scope in the future). Otherwise, they are suspended. As additional scopes, edges, and declarations are introduced, suspended resolution constraints may become solvable. Therefore, name resolution is specified as a global consistency condition rather than an eager operation. Moreover, these semantics allow a type checker to deal with non-lexical scoping such as forward references. We can issue a constraint that a reference should be resolvable before we create a declaration for that name. The constraint should be delayed until we encounter the declaration.

2.4.1 Statix

Statix is a constraint-based language used in the Spoofax language workbench for specifying type systems and name resolution declaratively. It builds on scope graphs to provide a unified framework in which both binding structures and typing constraints are expressed as logical constraints.

In Statix, programs are analyzed by generating and solving constraints over terms and scope graphs. Terms represent syntactic entities of the object language, while constraints capture relationships between them, such as equality, typing requirements, or name resolution conditions. Constraint solving proceeds incrementally, refining both the term structure and the scope graph until a consistent solution is found.

The principal design choice in Statix is that terms are first-order: they are constructed from user-defined constructors and variables, but do not include higher-order constructs such as lambda abstractions. As a result, binding structure is not represented within terms themselves, but externally via scope graphs. Likewise, unification in Statix is restricted to first-order terms. This separation enables flexible modeling of complex scoping mechanisms, including non-lexical scoping.

2.5 Another way to model name binding: Higher-Order Abstract Syntax

An alternative approach to modeling name binding is given by Higher-Order Abstract Syntax (HOAS), in which binding constructs of an object language are represented using the binding mechanisms of a meta-language (host language). Instead of representing scopes, declarations, and references explicitly, HOAS encodes binding structure directly using functions of the meta-language. The key idea is to represent object-language binders as meta-level functions. A bound variable is represented by a meta-level variable introduced by a function argument, and occurrences of that variable are represented by the same meta-level variable. As a result, substitution and α-equivalence are handled implicitly by the meta-language, without requiring explicit definitions of these operations.

For example, a λ-abstraction in the object language can be represented as a meta-level function wrapped in `Lam` constructor:

$$[\![\lambda x.\ body]\!] \Rightarrow \mathsf{Lam}(\mathsf{fun}\ x \Rightarrow body)$$

Here, x is a meta-level variable representing the bound variable, and $body$ is a term in which x may occur. Application is represented as:

$$[\![f\ x]\!] \Rightarrow \mathsf{App}(f, x)$$

In theory HOAS can be embedded in a variety of meta-languages, such as Agda or Coq, where binders are represented using higher-order functions. However, these systems rely on an intensional (or weak) function space, in which functions are treated as opaque values that cannot be inspected or decomposed by pattern matching. Moreover, when doing HOAS in a general purpose language, it is possible to write non-normalizing terms which the type checker will accept[1]. These problems can be reduced by implementing a variant of HOAS called PHOAS[2][5], but which sacrifices a lot of the convenience of working with HOAS terms.

For this purpose, logic programming languages are easier to use with HOAS. λ-Prolog and its derivations use the so-called λ-tree syntax [6], in which the user can define lambda terms, quantify variables and where the runtime manages fresh variables through eigenvariables. Contrasting to general purpose languages, the semantics of a logic programming language are slightly different. Programs are expressed as relations rather than functions, and evaluation proceeds by attempting to prove goals using unification and/or backtracking. To make the discussion more concrete, we will focus on an actual implementation of these ideas. ELPI [8] is an embeddable implementation of λ-Prolog that adopts the λ-tree approach. It uses higher-order pattern unification on meta-level representations of object-language terms, including those containing binders. Unification is restricted to the higher-order pattern fragment, where meta-variables may only be applied to distinct bound variables, to preserve decidability. A typical use case in ELPI is the representation of typing rules. For example, the typing rule for lambda abstraction can be encoded as:

```
check (lam Body) (arrow A B) :-
  (pi x \ type_of x A => check (Body x) B).
```

In this encoding, x is introduced as a fresh meta-level variable, by the use of the quantifier `pi` (for-all). The implication operator `=>` extends the context with the assumption that x has

[1] Unless the datatype of terms of the object language isn't positive, but in that case the program won't compile at all.

[2] Parametric Higher Order Syntax

type A when checking the body. The meta-language ensures that x is correctly scoped, and that occurrences of x in $Body$ refer to the same entity. Moreover, application can be expressed as:

```
infer (app F X) B :-
  infer F (arrow A B),
  check X A.
```

This representation contrasts with first-order encodings, where explicit environments or name management must be introduced. In HOAS, these aspects are delegated to the meta-language. Therefore, the difference in how binding structures can be inspected and manipulated greatly affects the design of type checkers and unification algorithms.

Chapter 3

Higher order unification

In this chapter we present the theory behind higher-order unification (HOU) as it is used inside the elaborator of a dependently typed language. We first motivate why a usable dependent type system cannot avoid inference, and why inference in turn forces us to solve unification problems over open terms that may contain unknowns under binders. We then survey the cases that arise during type checking which give rise to such problems, characterize their general shape in terms of *rigid-rigid*, *flex-rigid*, and *flex-flex* constraints, and explain why metavariable solutions must be treated as functions over a context. Finally, we describe Miller's *pattern fragment* as the decidable, well-behaved subset of HOU, and discuss strategies for handling constraints that fall outside the fragment.

3.1 Motivation for having HOU in the language

With dependent types users end up writing very complex types in places where it is not strictly needed. Inference is very powerful in this type system and it is crucial for a good user experience. Inference is triggered via underscores _, which can be inserted by the user or the elaborator component. An _ is an unfilled term, which we should be able to complete from constraints that we encounter in type checking.

Concretely, every underscore is replaced during elaboration by a fresh *metavariable* `?M`, and type checking proceeds with the term that contains it. Whenever a typing rule reduces to checking that two open terms must be equal, those metavariables become unknowns in a *unification problem*. For instance, given the polymorphic identity

$$\mathrm{id} : (A : \mathrm{Type}) \to A \to A,$$

a user that does not want to spell out the type argument writes id _ 42. The elaborator replaces the underscore by a fresh metavariable, obtaining id $?M$ 42, and the typing rule for application generates the constraint $?M \doteq \mathbb{N}$. A solver that returns $?M := \mathbb{N}$ then completes the term automatically.

Crucially, the constraints we encounter are not always between closed first-order terms. Because types may depend on values, a metavariable can occur in a position where its solution must mention bound variables introduced by surrounding λ- or Π-binders. The unknown therefore stands for a *function*, and the corresponding unification problem is higher-order. The language must consequently be equipped with a higher-order unification (HOU) procedure to drive inference.

3.2 Cases in which type checking requires HOU

In [15], Martin-Löf introduced the equality type with a single element that will only type check if the arguments to the equality type are definitionally equal. In the following section,

we will discuss how equality requires unification using Agda. Agda is well suited for our examples because its elaborator represents missing information using metavariables and solves the resulting constraints through higher-order pattern unification primarily. The process is also relatively transparent, as users can inspect unresolved metavariables and the constraints associated with them and even interact ([22]) with them by asking the language to infer values one at a time.

In Agda the equality type is written as $a \equiv b$, with its sole inhabitant being the reflexivity proof $\text{refl} : (A : \text{Type})\,(x : A) \to x \equiv x$. Suppose the user is constructing a proof whose expected type they leave partially open by writing an underscore on one side:

$$\text{refl}\ \mathbb{N}\ 42\ :\ f\ _\ \equiv\ f\ 42.$$

The underscore is elaborated to a fresh metavariable $?M$, giving the expected type $f\ ?M \equiv f\ 42$. The typing rule for refl demands that both sides of the equation be definitionally equal, which becomes the unification problem $f\ ?M \doteq f\ 42$. Decomposing on the rigid head f yields another constraint $?M \doteq 42$, which is immediately solvable.

A more revealing example arises as soon as an underscore is written under a binder. Consider a vector lookup function whose return type is left to inference:

Listing 3.1: Higher-order constraint arising from an underscore under a binder

```
lookup : (A: Set) -> (n: Nat) -> (xs: Vec A n) -> (i: Fin n) -> _
lookup A n xs i = ...
```

The trailing underscore appears in the scope of `A`, `n`, `xs`, and `i`, so the elaborator must introduce a metavariable $?M$ whose telescope contains those variables. When the body of `lookup` is checked and found to return a value of type A, this produces a constraint of the shape $?M\ [A\ n\ xs\ i] \doteq A$. The constraint cannot be solved by a closed term since it needs to use A. Constraints where a metavariable is in the head position and needs bound variables in its solution are the higher-order problems that HOU handles.

3.3 Metavariable solutions are functions

The higher order problems require metavariables to be able to use their context. In the literature there are 2 ways to view metavariables: closures or lambda abstractions. Metavariable closures [20] are used to represent an occurrence of the metavariable, and consists of the metavariable along with a suspended substitution from the context in which it was created to the context in which it occurs. The lambda abstraction representation is slightly easier to reason about and does not introduce additional terms in the language. In this form, metavariables are always functions and upon creation they are immediately applied to the distinct bound variables in scope. Therefore, a metavariable in context $?M\ [A\ n\ xs\ i]$ becomes $?M\ A\ n\ xs\ i$. This is the representation as introduced by [17] and will be used in this chapter.

Miller also formalized in [17] the three cases of possible unification problems: rigid-rigid, flex-rigid and flex-flex. To make this precise, we fix the convention that every term is written in *spine form* $h\ \vec{e}$, where the *head* h is either a constant, a bound variable, or a metavariable, and $\vec{e}$ is a (possibly empty) sequence of arguments. A head is *rigid* when it is a constant or bound variable (or in general cannot be changed by a metavariable being solved), and *flex* when it is a metavariable. Each metavariable $?M$ is introduced in a typing context $\Gamma = x_1 : A_1, \ldots, x_n : A_n$ and carries a *telescope* declaring the variables it may mention. Its solution is therefore a closed term of type $\Pi\, \vec{x : A}.\, B$ that is applied to some spine of arguments at every occurrence. From this point on we identify a metavariable with the function it represents.

A unification problem has the form $\Gamma \vdash t_1 \doteq t_2$. The three cases that arise are classified by the heads of the two sides:

Rigid-rigid. Both sides have a rigid head. The constraint is solvable if and only if the heads are the same and their argument spines are unifiable. We write this as the standard decomposition rule:

$$\frac{\begin{array}{cc} h = h' & \Gamma \vdash e_1 \doteq e_1' \\ \cdots & \Gamma \vdash e_n \doteq e_n' \end{array}}{\Gamma \vdash h\ e_1 \cdots e_n \ \doteq\ h'\ e_1' \cdots e_n'}\ \textsc{rr-decomp}$$

If $h \neq h'$, the constraint is unsolvable and the elaborator must report a type error. For example, cons 1 nil $\doteq$ cons $?M$ nil decomposes to $1 \doteq ?M$ and is solvable by $?M := 1$, whereas cons 1 nil $\doteq$ nil fails on a rigid head mismatch.

Flex-rigid. Exactly one side has a metavariable as its head, $?M\ \vec{e} \doteq t$ with t rigid. The unifier must *invert* the application and find a closed function $\lambda \vec{x}.\, s$ such that substituting it for $?M$ reduces the equation to a tautology. In general inversion is non-trivial because $?M$ may be applied to non-bound variables, multiple occurrences of the same variable, or other metavariables. We will return to this case in section Section 3.4.

Flex-flex. Both sides have metavariable heads: $?M\ \vec{e} \doteq ?N\ \vec{e'}$. Such constraints generally admit infinitely many solutions; for example $?M\ x \doteq ?N\ x$ is satisfied by any common refinement of $?M$ and $?N$. Without further information the elaborator cannot commit to one of them, which is a source of the undecidability of general HOU.

3.4 Additions to higher order unification and the pattern fragment

General HOU is undecidable because there exists problems that admit infinitely many solutions. However, Miller discovered that problems in the pattern fragment always have a most general unifier (which is also unique). This gives rise to 2 methods of doing HOU with pattern fragment. If a problem isn't in the pattern fragment, you have the option to immediately fail or delay the problem until further solutions simplify it.

3.4.1 The pattern fragment

A unification problem $\Gamma \vdash ?M\ \vec{e} \doteq t$ is said to be in the *pattern fragment* when the spine $\vec{e}$ consists of pairwise distinct bound variables (up to η-expansion). The same condition is required of every metavariable application appearing in t. Miller's central result [17] is that within this fragment unification is decidable and, whenever a unifier exists, it is the unique most general one. The corresponding solving rule is direct:

$$\frac{\begin{array}{c} x_1, \ldots, x_n \text{ distinct bound vars} \\ \mathrm{FV}(t) \subseteq \{x_1, \ldots, x_n\} \qquad ?M \notin t \end{array}}{\Gamma \vdash ?M\ x_1 \cdots x_n \ \doteq\ t \ \rightsquigarrow\ ?M := \lambda x_1 \cdots x_n.\, t}\ \textsc{pat-flex-rigid}$$

The three imposed conditions of the pattern fragment correspond to the obstacles encountered in general HOU:

1. distinct bound variables make the substitution $[x_i \mapsto x_i]$ trivially invertible
2. the free-variable check (sometimes called a *scope check*) ensures the proposed solution does not mention variables outside $?M$'s telescope
3. the occurs check ($?M \notin t$) prevents constructing a cyclic solution.

Flex-flex problems within the fragment are handled similarly, by introducing a fresh metavariable defined over the intersection of the two variable lists [2].

3.4.2 Outside the pattern fragment

A problem is outside the pattern fragment in cases such as: the spine of a metavariable may contain a non-variable term (e.g. $?M\ 0$), a repeated variable ($?M\ x\ x$), or an argument whose head is itself an unsolved metavariable. When this happens the elaborator faces two design choices:

1. **Eager failure:** Reject the program immediately. This is simple to implement and predictable, but it forces users to supply otherwise inferable terms whenever the constraint generator happens to produce a non-pattern problem.

2. **Postpone:** Suspend the constraint and continue elaboration. Later constraints may instantiate the offending metavariables, and after substitution the suspended problem may fall back into the pattern fragment and become solvable. This *dynamic pattern unification* strategy [2] is the approach taken by elaborators such as those of Agda and Lean.

As an illustration, $?M\ x\ x \doteq x$ is not in the pattern fragment because of the repeated variable, and it admits the two solutions $?M := \lambda x\, y.\, x$ and $?M := \lambda x\, y.\, y$. Eager failure would reject the program since it is not in pattern fragment. If we postpone, a separate constraint might later force $?M := \lambda x\, y.\, y$ and the suspended problem reduces to $x \doteq x$ and succeeds.

3.4.3 Pruning

Pruning is triggered by a flex-rigid problem $?M\ \vec{x} \doteq t$ whose left-hand side is in pattern form, but where the scope condition of PAT-FLEX-RIGID fails because t contains a variable that is not in $\vec{x}$. The naive inversion $?M := \lambda \vec{x}.\, t$ would be ill-scoped. However, if the offending variable enters t only through the spine of *another* metavariable $?N$, we can repair the problem by refining $?N$ so that it is no longer allowed to depend on that variable. After the refinement, the variable disappears from t and the original constraint becomes solvable by PAT-FLEX-RIGID.

Concretely consider the context $\Gamma = a, b$ with two metavariables

$$?M : \mathbb{N} \to \mathbb{N}, \qquad ?N : \mathbb{N} \to \mathbb{N} \to \mathbb{N},$$

where $f : \mathbb{N} \to \mathbb{N}$ is a known constant. Consider the constraint

$$?M\, a \ \doteq\ f\, (?N\, a\, b).$$

The spine $[a]$ of $?M$ is a single distinct bound variable, so the pattern check on the left-hand side passes. The right-hand side, however, mentions b via the term $?N\, a\, b$, and $b \notin \{a\}$. Inverting directly would produce $?M := \lambda a.\, f\, (?N\, a\, b)$, but the free occurrence of b escapes $?M$'s telescope and the substitution is ill-formed. The key observation is that b enters t only through $?N$'s second argument. If we forbid $?N$ from depending on its second argument at all, the term $?N\, a\, b$ will no longer mention b. We can therefore introduce a fresh metavariable

$$?P : \mathbb{N} \to \mathbb{N}$$

and refine $?N$ by the partial solution

$$?N \ :=\ \lambda x\, y.\, ?P\, x.$$

This makes $?N$ ignore its second argument, and leaves it solvable for when $?P$ gets instantiated. Substituting throughout, $?N\, a\, b$ reduces to $?P\, a$, and the original constraint becomes

$$?M\, a \ \doteq\ f\, (?P\, a),$$

which is in the pattern fragment as FV($f\ (?P\ a)) = \{a\} \subseteq \{a\}$ and the occurs check holds. We can apply PAT-FLEX-RIGID to obtain $?M := \lambda a.\ f\ (?P\ a)$. The pruning of $?N$ was enough to bring the problem back inside Miller's fragment, without committing to a final value for $?P$.

Pruning generalizes to nested metavariable applications and interacts subtly with dependent types. A prune that would make a downstream constraint ill-typed must be rejected rather than performed, and in some cases there is no valid prune at all as shown in [2]. Additionally, in practice pruning works better with postponement, since a successful prune can unblock constraints that drive new prunes or new pattern-fragment solutions in turn.

Chapter 4

A Language with Dependent Types in Statix

4.1 Language

The language implemented in Statix showcases the powerful features of dependent types. The core of the language is a lambda calculus with Pi types and let bindings. Concretely the syntax of our language is defined using the SDF3 syntax:

Listing 4.1: Necessary syntax for dependent types

```
Expr.Let = [let [ID] = [Expr]; [Expr]]
Expr.Set = "Set"
Expr.Var = ID
Expr.Pi = "(" ID ":" Expr ")" "->" Expr {right}
Expr.Lam = "\\" ID "." Expr
Expr.App = Expr Expr {left}
Expr.Ann = Expr "::" Expr
```

The language is formed from lambda abstractions, applications, variables, let bindings, pi types and type annotations. The latter are very important for our bidirectional type checking approach [9]. In this method a type checker has two routines: *infer* and *check*. Variables, let expressions and applications are inferable, while lambda abstractions need to be checked against a type. Type annotations mark the transition from infer mode to check mode. Lastly, the `Set` constructor represents the type of types. In our language, `Set` has type `Set`. This allows Girard's paradox [13], which shows that self-referential universes can derive a contradiction. However, the typing of `Set` is implemented in the language without loss of generality and can easily be extended with universe levels like in Agda.

The two routines required for bidirectional typing have the following signatures:

```
infer : scope * Expr -> (Expr * Expr)
check :  scope * Expr * Expr -> Expr
```

Both infer and check take the current scope and an expression from the object language. `check` also takes a type as an additional argument. Normally, infer would return the inferred type, while check wouldn't return anything since it would either succeed or fail. The extra `Expr` comes from the fact that both relations *refine* their input terms: variables get replaced with fresh variables with globally unique names. In chapter 5 we will also show how placeholders "_" are refined to metavariables. Lastly, the following example represents a valid program in our language:

```
(\T.\x. x) :: (T : Set) -> (t : T) -> T
```

We are given an annotated lambda term `\T.\x.x` which we can check against `(T : Set) → (t : T) → T`. The example will type check since we know that T is a type, x has type T and the return types match.

Typing rules. In the following we will formalize the type checking rules for the language. Since we are working within a bidirectional framework we will use the notation $e \Uparrow T \rightsquigarrow e'$ to express that term e is inferred to type T and can be refined to term e'. Furthermore, $e \Downarrow T \rightsquigarrow e'$ works similarly to the previous rule, but term e is checked against a given type T. We also require a symbol to express equality between two terms: $x \equiv y$.

Inferred terms

$$\frac{s \vdash a \Uparrow A \rightsquigarrow a' \quad (x_{fresh}, A) :: s \vdash b \Uparrow T \rightsquigarrow b'}{s \vdash (\mathsf{let}\ x = a;\ b) \Uparrow T \rightsquigarrow (\mathsf{let}\ x_{fresh} = a';\ b')}\ \textsc{Let}$$

$$\frac{s \vdash \mathsf{getTy}(s, v) = T \quad s \vdash \mathsf{declaredID}(s, v) = v'}{s \vdash Var(v) \Uparrow T \rightsquigarrow Var(v')}\ \textsc{Var} \qquad \frac{s \vdash}{s \vdash Set \Uparrow Set \rightsquigarrow Set}\ \textsc{Set}$$

$$\frac{s \vdash f \Uparrow (x : A) \to B \rightsquigarrow f' \quad s \vdash e \Uparrow A' \rightsquigarrow e' \quad A' \equiv A \quad [x \mapsto e']s \vdash B \Rightarrow_\beta R}{s \vdash fe \Uparrow R \rightsquigarrow f'e'}\ \textsc{App}$$

$$\frac{s \vdash T \Downarrow Set \rightsquigarrow T' \quad s \vdash e \Downarrow T' \rightsquigarrow e'}{s \vdash \mathsf{Ann}(e, T) \Uparrow T' \rightsquigarrow \mathsf{Ann}(e', T')}\ \textsc{Ann}$$

Checked terms

$$\frac{s \vdash A \Downarrow Set \rightsquigarrow A' \quad (x_{fresh}, A') :: s \vdash B \Downarrow Set \rightsquigarrow B'}{s \vdash (x : A) \to B \Downarrow Set \rightsquigarrow (x_{fresh} : A') \to B'}\ \textsc{Pi}$$

$$\frac{(x, A) :: s \vdash b \Downarrow B \rightsquigarrow b'}{s \vdash \backslash x.b \Downarrow (x : A) \to B \rightsquigarrow x_{fresh}.b'}\ \textsc{Lam} \qquad \frac{s \vdash e \Uparrow T' \rightsquigarrow e' \quad T' \equiv T}{s \vdash e \Downarrow T \rightsquigarrow e'}\ \textsc{Unknown}$$

4.2 Variables and substitution

As outlined in chapter 2, name binding in dependent types is challenging in the context of type checking. A type checker needs to store a context (information about variables and their **types**) in addition to performing beta reduction, which requires substitution of **values**.

A context is represented as an ordered list of associations between names and types:

$$\Delta = (x, A) :: \Delta \qquad x : A, A : Set$$

Entries in the context represent the declaration of a variable, because we can infer a variable's type when it is introduced in the scope. Later references of that variable name should resolve to its declaration entry. In languages with name shadowing a context will carry entries that have the same variable names. Then the question is to which entry should a reference resolve to. In the lexical scoping discipline a reference should resolve to the first entry in the context (added latest) that has the same name as itself. Our implementation will emulate this behavior using scope graphs.

In Statix we will implement lexical scoping by creating descendant scopes that reference their parent via **P** edges. For example, the following program will generate a scope graph:

Listing 4.2: let introduces a scope

```
let x = Set;
x :: Set
```

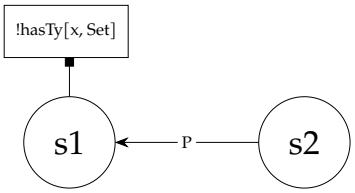


Figure 4.1: Scope graph of listing 4.2

When the checker infers the type of a variable it will add a `hasTy` relation to the scope graph. In Statix, later references of the variable should resolve to this entry using the `getTy` rule:

$$\frac{(x = id), P^* \vdash s \overset{hasTy}{\mapsto} (id, ty)}{\mathsf{getTy}(s, x) = ty}\ \textsc{get-Ty}$$

This scope graph query is presented similarly to the "Scopes as types" paper [4], where scope graph queries were formalized. The query starts from scope s and follows paths described by the regular expression P^* searching a relation labelled hasTy for an identifier id with the condition that it is the same as x. If such a declaration is found, the associated type ty is returned. The same mechanism can account for lexical nesting, shadowing, imports, and other name-resolution features while preserving the connection between variables in the program and their declarations.

Substitutions Besides keeping track of the typing context, the checker must also store substitutions. Consider a continuation of example 4.2, where `x` is used in a type:

```
let x = Set;
\a. a :: Set -> (\T. T) x
```

This example is valid in the lambda calculus with pi types, and the checker has to perform type level computation. First it should apply the lambda function `\T. T` on the variable `x` giving back `x` again. Then it should substitute `Set` for `x`.

To implement reduction we evaluated 3 strategies:

1. Perform substitution eagerly. In program 4.2, after `x` is type checked, it can be substituted in later expressions for `Set`. This approach is easy to implement but has the disadvantage of being very inefficient. On the one hand, if the variable is substituted for a more complex expression we would have to type check the whole expression again, wherever the variable appeared. And on the other hand, we need to traverse and change the whole AST to the leaves. Besides being costly, it also interacts negatively with Statix, which can no longer relate errors in the checking process with the location where they happen in the surface code.

2. Normalization by evaluation (NbE) [1], which is a technique that works by converting terms to values and viceversa. NbE is a widely used technique, but fitting it into Statix's first order semantics is challenging. Specifically, lambda terms in the object language are usually converted to a value that holds an actual function in the meta language, in order to have substitution for free using the meta language's semantics. However, Statix has no functions or applications so implementing NbE doesn't solve the problem of manually performing substitution.

3. Call by name [25] semantics, while storing substitutions explicitly in the scope graph. In call by name, we associate a name with a term, but do not perform any operations eagerly. When the name is referenced we query if it has any substitutions which are visible from the current scope. Doing this is Statix requires associating names with closures[1] in the scope graph.

The third strategy was chosen as it integrates the best with Statix's semantics. Explicitly storing substitutions avoids unnecessary work, and it has been implemented in other type checkers using Statix [4, 14]. In this strategy the type checker stores substitutions via a separate relation `subst`, which pairs an `ID` with a `Closure(scope, Expr)`.

$$\mathsf{subst} : \mathsf{ID} \rightarrow \mathsf{Closure}$$

The closure might contain free variables that also have `subst` relations in other scopes. The reduce function will have to recursively evaluate these substitutions. The primitive that we designed in Statix is lookupSubst defined as follows:

$$\frac{(x = id), P^* \vdash s \overset{subst}{\mapsto} (id, Closure(s', t))}{lookupSubst(s, x) = Closure(s', t)} \text{ LOOKUP-SUBST}$$

Name capture. A very complicated problem to solve which is the source of many bugs in type checking is capture-avoiding substitution. This occurs when we perform substitutions of the form $e[x := t]$, where free variables in t are actually bound in e. Consider an example where we perform naive substitution:

$$\texttt{(\textbackslash x.\textbackslash y.x) y} \Rightarrow_\beta \texttt{\textbackslash y.y}$$

Syntactically[2], the free y and the one used as binder in the lambda expression have the same name. The term we obtain from naive substitution would be wrong since the y in the body refers to the binder instead of the original free variables.

The free variables cannot change their names, but bound variables can. It is possible to rename a binder when we want to inspect or perform substitution on the body. The previous example should be solved by:

$$\texttt{(\textbackslash x.\textbackslash y.x) y} \equiv \texttt{(\textbackslash x.\textbackslash z.x) y} \Rightarrow_\beta \texttt{\textbackslash z.y}$$

There are many ways we can solve this in a type checker. Most solutions involve representing variables differently from their syntactic form. This of course involves a pre-processing or conversion of the terms. One of the most known practices is to use De Bruijn indices for bound variables. In this representation, variables that are bound do not use their syntactic name, but rather an integer that expresses the distance[3] to their bind point. Free variables can use names, but there are ways to convert them to De Bruijn syntax as well.

$$\texttt{(\textbackslash x.\textbackslash y.x) y} \equiv \texttt{(\textbackslash _.\textbackslash _.}\hat{1}\texttt{) y}$$

Here, $\hat{1}$ refers to the binder one level outside the innermost lambda. Since bound variables no longer depend on names, alpha-equivalent terms have the same representation, and capture-avoiding substitution can be implemented by shifting indices rather than by generating fresh names. However, De Bruijn indices do not interact well with the rest of our implementation. Statix is based on name resolution through scope graphs: names in the syntax are connected

[1] A closure in this context means storing an expression along with a scope. Any free variable in the expression should have a declaration in that scope (or parent scopes)

[2] In the sense of their surface language representation

[3] 0 is used for the immediate binder, then 1 for the next and so on

to declarations through resolution paths. These names are also the link between the internal analysis and editor services such as go-to-definition and error reporting. If all bound variables are translated to nameless indices before type checking, the checker loses the syntactic names that Statix is designed to reason about. This would make the implementation less natural in Statix and would require an additional mapping between nameless internal terms and the original source-level names.

Instead, our implementation keeps names in the term language, but distinguishes between two kinds of variable identifiers. User-written variables are represented by their syntactic names `Loc(string)`. Internally generated variables are represented by a `Fresh(scope, string)` constructor, which associates the syntactic name with the scope in which it was declared. Thus, the implementation can preserve the source-level names needed by Statix while still avoiding accidental capture during substitution. Two syntactic variables with the same name will be converted to different fresh variables **iff** their declarations are different (are in different scopes).

Before a term is checked or inferred, the binders relevant to that term are freshened. Freshening replaces a syntactic binder name by a fresh internal name, and occurrences bound by that binder are made to refer to the fresh name. For example, the term

```
\Loc(y). Loc(y)
```

will be internally represented as

```
\Fresh(s, y). Fresh(s, y)
```

where `Fresh(s, y)` is a fresh name generated for the binder. The printed name may still be `y`, but the identity used by the type checker is unique. Consequently, a substitution should never insert a term containing a free syntactic variable `Loc(y)`. `Loc` variables can be captured by binders. In order to resolve the declaration of a `Loc` variable, the scope graph queries `lookupSubst` and `getTy` match the name with the closest lexical match, which should be a `Fresh` variable already. In the Statix code we account for this by adding another case for the two queries:

$$\frac{(x = \mathsf{toString}(id)), P^* \vdash s \overset{hasTy}{\mapsto} (id, ty)}{\mathsf{getTy}(s, Loc(x)) = ty}\ \textsc{get-Ty-Loc}$$

$$\frac{(x = \mathsf{toString}(id)), P^* \vdash s \overset{subst}{\mapsto} (id, Closure(s', t)}{\mathsf{lookupSubst}(s, Loc(x)) = Closure(s', t)}\ \textsc{lookup-Subst-Loc}$$

Here `toString` obtains the original string name from `Loc` or `Fresh` constructors. With this representation, the earlier example is handled without relying on De Bruijn indices:

```
(\x.\y.x) y ⇝ (\Fresh(s1, x).\Fresh(s2, y).Fresh(s1, x)) Fresh(s3, y)
            ⇒β \Fresh(s2, y).Fresh(s3, y)
```

The binder `y` and the free variable `y` are converted to different identifiers, even though they may have the same surface spelling. Therefore, the free variable remains free after substitution.

4.3 Beta reduction

For the type checker we require 2 important routines: reducing terms to weak head normal form (whnf) and to their normal form.

Whnf. A term is in weak head normal form when no reduction is possible at its head position. Reduction does not continue under lambda abstractions, and it does not reduce arguments unless they are needed by a head computation. For example, $(\lambda x.\ x)\ Set$ is not in weak head normal form, because its head is a beta-redex that reduces to Set which is in weak head normal form. By contrast, $\lambda x.\ ((\lambda y.\ y)\ x)$ is already in weak head normal form, because the outermost constructor is a lambda abstraction. The beta-redex inside the body is not reduced when computing weak head normal form. In Statix weak head normalization was implemented as a head-reduction procedure with an explicit argument spine. We write

$$\mathsf{whnf}_h(s; e; \overline{a}) = (s', e')$$

to mean that expression e, evaluated in scope s, reduces to weak head normal form e' under scope s', while carrying a pending argument spine $\overline{a}$. Each argument in the spine is a closure (s_i, e_i), where s_i is the scope in which the argument e_i must be interpreted.

The ordinary weak head normalization function is the special case with an empty argument spine:

$$\mathsf{whnf}(s; e) = \mathsf{whnf}_h(s; e; \epsilon).$$

$\mathsf{whnf}_h(s; e_1\ e_2; \overline{a})$	$=$	$\mathsf{whnf}_h(s; e_1; (s, e_2) :: \overline{a})$	Wh-App
$\mathsf{whnf}_h(s; (e : A); \overline{a})$	$=$	$\mathsf{whnf}_h(s; e; \overline{a})$	Wh-Ann
$\mathsf{whnf}_h(s; \textbf{let}\ x = v\ \textbf{in}\ b; \overline{a})$	$=$	$\mathsf{whnf}_h(s[x' \mapsto (s, v)]; b; \overline{a})$ where $x' = \mathsf{fresh}(s, x)$	Wh-Let
$\mathsf{whnf}_h(s; \lambda x.\ b; (s_a, a) :: \overline{a})$	$=$	$\mathsf{whnf}_h(s[x' \mapsto (s_a, a)]; b; \overline{a})$ where $x' = \mathsf{fresh}(s, x)$	Wh-Beta
$\mathsf{whnf}_h(s; \lambda x.\ b; \epsilon)$	$=$	$(s, \lambda x.\ b)$	Wh-Lam
$\mathsf{whnf}_h(s; H; \overline{a})$	$=$	$\mathsf{rebuild}((s, H), \overline{a})$ $H \in \{\Pi x : A.\ B,\ A \to B,\ \mathsf{Set}\}$	Wh-Head
$\mathsf{whnf}_h(s; x; \overline{a})$	$=$	$\mathsf{whnf}_h(s_v; v; \overline{a})$ if $\mathsf{lookupSubst}(s, x) = (s_v, v)$	Wh-VarSubst
$\mathsf{whnf}_h(s; x; \overline{a})$	$=$	$\mathsf{rebuild}((s, x), \overline{a})$ if $\mathsf{lookupSubst}(s, x)$is undefined	Wh-VarStuck

The first four rules perform head computation. Applications are decomposed into a head and an explicit argument spine. Type annotations are ignored during reduction. Let-bindings extend the current scope with a delayed substitution, and lambda abstractions consume one pending argument by extending the current scope with a delayed substitution for the bound variable. If a lambda has no pending argument, it is already in weak head normal form.

The remaining rules describe stuck heads. A Π-type and Set do not reduce at the head position, so the pending spine is rebuilt. Variables reduce only when they have a delayed substitution in the current scope. Otherwise, they are neutral terms, and the spine is rebuilt around them. rebuild reconstructs applications once head reduction becomes stuck:

$$\begin{aligned} \mathsf{rebuild}((s, e), \epsilon) &= (s, e) \\ \mathsf{rebuild}((s, e), a :: \overline{a}) &= \mathsf{rebuild}(\mathsf{apply}((s, e), a), \overline{a}). \end{aligned}$$

Applying a head to an argument preserves the argument's original scope. If the head and argument are already in the same scope, the application can be rebuilt directly:

$$\mathsf{apply}((s, e), (s, a)) = (s, \mathsf{App}(e, a)).$$

If the argument comes from a different scope or we have made some progress evaluating it, the argument is stored as a fresh temporary substitution, and the rebuilt application refers to the temporary variable:

$$\mathsf{apply}((s, e), (s_a, a)) = (s[t \mapsto (s_a, a)],\ \mathsf{App}(e, t)) \qquad \text{where } t = \mathsf{fresh}(_, _app).$$

Normalize. Full normalization extends weak head normalization in two steps. First, it reduces the input expression to weak head normal form. Then, once the outermost constructor is known, it recursively normalizes the immediate subexpressions of that constructor. Thus, unlike weak head normalization, full normalization continues under lambda abstractions, inside applications, and inside function types. We write

$$\mathsf{normalize}(s, e) = e'$$

to mean that expression e, evaluated in scope s, fully normalizes to e'. The auxiliary judgment

$$\mathsf{normalize}_{\mathsf{wh}}(s, e) = e'$$

normalizes an expression that has already been reduced to weak head normal form.

$$\frac{\mathsf{whnf}(s, e) = (s', w) \qquad \mathsf{normalise}_{\mathsf{wh}}(s', w) = n}{\mathsf{normalise}(s, e) = n}\ \textsc{Norm-Whnf}$$

$$\frac{\mathsf{normalise}(s, A) = A' \qquad \mathsf{normalise}(s, B) = B'}{\mathsf{normalise}_{\mathsf{wh}}(s, \Pi x : A.\ B) = \Pi x : A'.\ B'}\ \textsc{Norm-Pi}$$

$$\frac{\mathsf{normalise}(s, b) = b'}{\mathsf{normalise}_{\mathsf{wh}}(s, \lambda x.\ b) = \lambda x.\ b'}\ \textsc{Norm-Lam}$$

$$\frac{\mathsf{normalise}(s, f) = f' \qquad \mathsf{normalise}(s, a) = a'}{\mathsf{normalise}_{\mathsf{wh}}(s, f\ a) = f'\ a'}\ \textsc{Norm-App}$$

$$\frac{}{\mathsf{normalise}_{\mathsf{wh}}(s, e) = e}\ \textsc{Norm-Neutral}$$

The checker does not always need to fully normalize expressions. In most typing rules, it only needs to inspect the outermost constructor of a type. For this reason, our implementation reduces expressions to weak head normal form before pattern matching on them. Converting to weak head normal form is cheaper than full normal form. An efficient type checker will prefer whnf which is enough to make progress in type checking and unification. Converting to full normal form is however useful in some particular cases. It is generally used to display the type of an identifier or expression. In our infer routine it is also used when computing the codomain of Pi types:

```
let f = ... :: (x : Nat) -> (y : Nat) -> (Fin x, Fin y)
f 1 2
```

In the example above, the resulting type from the application `f 1 2` is `(Fin 1, Fin 2)`. If we were to only convert to whnf, the type would be `(Fin x, Fin y)` together with a scope where x is bound to 1 and y to 2. We therefore prefer to substitute whatever was bound and return a type that doesn't need further reducing.

4.4 Equality

Equality is an important feature of dependent type languages. In most programming languages, equality is a built-in feature of the underlying compiler. With non-dependently typed languages, it can be extended by the user, most commonly by implementing the $==$ operator or some form of equality trait. In code, equality is often formulated as a test that two different values are equal, returning a boolean answer.

Equality type constructor. In contrast, dependently typed languages, such as Agda, model equality as formulated by Martin Löf. In his type theory [15], equality is a **type constructor** and not a function or operator. The equality type is defined as follows:

$$_ \equiv _ : (A : Set) \to (x : A) \to (y : A) \to Set$$

This type is well-formed for any two values that belong to the same type A. For brevity, the type A of the left and right-hand side will be left implicit. As an example, valid types we can form with the equality constructor are: $1 \underset{\mathbb{N}}{\equiv} 2$, $true \underset{\mathbb{B}}{\equiv} true$, but not $Set \equiv 1$. Note that we can write $1 \equiv 2$, even if 1 isn't equal to 2, and it will still be a valid type.

Elements of equality type. The type $x \equiv y$ is analogous to the proposition "x is equal to y". However, the proof for this proposition consists of producing an element of the type $x \equiv y$. In Martin Löf, there is a single way to construct an element of the equality type directly:

$$refl : (A : Set) \to (x : A) \to x \equiv x$$

Constructor refl stands for reflexivity and produces a proof that x is equal to itself. The important consequence is that equality proofs are checked against an expected equality type. The constructor refl can only inhabit an equality type whose two sides are **definitionally equal**. For example, refl can prove $1 \equiv 1$, and it can also prove $(\lambda x.\ x)\ 1 \equiv 1$, since the left-hand side reduces to 1. This separates propositional equality, represented by the type $x \equiv y$, from definitional equality, which is the type checker's internal notion of equality up to reduction. Furthermore, propositional equality cannot be proven just by refl. A propositional equality proof will make use of the equality eliminator J [15], which allows case analysis on a proof.

Equality in Statix. In our language, equality is implemented closely to the Martin-Löf theory. The equality type constructor is represented by `==` and takes two terms (e.g. `Set == Set`). When checking that the type is valid, we also check that the terms on the left and right-hand side have the same types. Reflexivity is built-in the language, with the keyword `refl` being part of the syntax. `refl` is a checked term where the type must be $x \equiv y$, and x must unify with y. This approach is different from Martin-Lof theory, where `refl` takes the

element as argument. We illustrate the semantics of equality in our bidirectional approach with the following rules:

$$\frac{s \vdash x \Uparrow A \rightsquigarrow x' \quad s \vdash y \Downarrow A \rightsquigarrow y'}{s \vdash x == y \Downarrow Set \rightsquigarrow x' == y'}\ \textsc{Eq} \qquad \frac{s \vdash x \equiv y}{s \vdash \mathsf{refl} \Downarrow x == y}\ \textsc{Refl}$$

We can model `refl` in this way without loss of generality, since its arguments are implicit (can be inferred from the type that refl is checked against). As an example consider the following code:

Listing 4.3: definitional equality checking in Statix

```
let proof = (\x. refl) :: (x : Set) -> (x == x);
proof
```

In order to type check this example, we have to show that the x we introduced is equal to itself. `refl` will trigger only the unification between x and x, which will hold trivially.

4.5 Statements and forward references

In the sections above, we have described all the terms in our language. Terms are used to write expressions, which is the syntactic unit that we can evaluate and type check. We introduce statements in the language to simplify certain aspects of name binding for the user. For instance, using only expressions to define a function with a body and a signature is cumbersome:

```
let f = (\x.\y. y) :: (x: Set) -> (y: x) -> x;
```

Instead users can create a *definition*. The grammar for statements is given in the following:

Listing 4.4: Syntax for statements

```
Start.Program = [[Stmt*] [Expr]]
Start.ProgramOnly = [[Stmt*]]

Stmt.Def = [def [ID] : [Expr]; [ID] = [Expr];]
Stmt.Postulate = [postulate [ID]: [Expr];]
```

We now consider a program to be formed from zero or more statements, optionally followed by a single expression. The available statements are definitions and postulates. A postulate attaches a type to a name, without having to provide a value for the name. Beta reduction will treat this name as irreducible. On the other hand, definitions attach a body (a term) and a type to a given name. Listing 4.5 shows how postulates and definitions can be used in our Statix implementation.

Listing 4.5: Example postulates and definitions

```
postulate Nat : Set;
postulate z : Nat;
postulate suc : Nat -> Nat;

def plusone : Nat -> Nat;
plusone = \n. suc n;
```

Scoping of statements. Statements as we have presented them above are analogous to top level constructs in other programming languages. In C, a top level construct would be the definition of a function or a type. In Java, you could consider a class to be top level. There is a significant difference in how language implementations treat scoping at the top level. If we consider the C language, then top level definitions can only refer to definitions declared before them, akin to lexical scoping. Contrary to C, Java allows **forward-references**. In a Java compilation unit, it does not matter in which order the user introduces definitions. A top level construct is allowed to reference other visible constructs that are declared after it. This scoping behavior is not trivial to implement with ad-hoc compilers. Usually it involves splitting the type checking in 2 phases. The first pass, will pre-process the signatures of the statements and add them to the scope. The final pass would iterate over the statements and type check their bodies, with all the other signatures available in scope.

Forward references with scope graphs. In our Statix implementation we have implemented forward references for statements. They make it easier for users to write statements in an order that makes sense for their use case, rather than an order established by the compiler. Using the properties of scope graphs and scope graphs queries we can implement this type of scoping without additional passes. Recall that the scope graphs framework is based on constraints. Our type checker emits constraints and scope graph queries, which are all managed by a solver. The solver will delay queries from a scope, until it is certain that its result will not change. The result of a query can change if we add name bindings to the target scope (or its parents).

We can make use of this behavior to delay the resolution of names when we haven't reached their declaration. Concretely, for statements we will use a single top level scope S_{stmt}. Statements are processed in the order given by the AST. If we are able to check a definition, we will update S_{stmt} with a `!hasTy` relation for the definition. Conversely, if we require the type of a name that we haven't processed yet, that query will be delayed and the checker will process further statements. After going through all declarations, Statix will be able to establish that S_{stmt} cannot be extended with bindings, and all delayed queries should resolve[4]. Consider the following program, where the body of `g` refers to `f`, `suc`, and `zero` before these names are declared:

Listing 4.6: Forward references between statements

```
1. postulate Nat : Set;

2. def g : Nat;
3. g = f (suc zero);

4. def f : Nat -> Nat;
5. f = \n. suc n;

6. postulate suc : Nat -> Nat;
7. postulate zero : Nat;
```

When the checker processes the first postulate, it adds a type binding for `Nat` to this scope. When it reaches the definition of `g`, it first checks the declared type `Nat`. This succeeds because the type of `Nat` is already available. The checker can then add the binding `hasTy`(g, Nat) to S_{stmt}. However, checking the body of `g` produces queries for the types of `f`, `suc`, and `zero`. At this point these names have not yet been declared, so the corresponding queries cannot be resolved immediately. Instead, the solver delays them. As the checker continues

[4]Either successfully or error

through the remaining statements, new bindings are added to S_{stmt}. The declaration of `f` adds $\texttt{hasTy}(f, Nat \to Nat)$, the postulate for `suc` adds $\texttt{hasTy}(suc, Nat \to Nat)$, and the postulate for `zero` adds $\texttt{hasTy}(zero, Nat)$. Once these bindings have been added, the delayed queries from the body of `g` can be resumed and resolved. The body `f (suc zero)` can then be checked against the expected type `Nat`.

The order in which the constraints are emitted is different from the order in which all queries are resolved. For Listing 4.6, the relevant events are:

line no.		constraint/query	
1	:	$\texttt{!hasTy}(Nat, Set)$	is added
2	:	$\texttt{!hasTy}(g, Nat)$	is added
3.1	:	$\texttt{getTy}(S_{\mathsf{stmt}}, f)$	is delayed
3.2	:	$\texttt{getTy}(S_{\mathsf{stmt}}, suc)$	is delayed
3.3	:	$\texttt{getTy}(S_{\mathsf{stmt}}, zero)$	is delayed
4	:	$\texttt{!hasTy}(f, Nat \to Nat)$	is added
5	:	$\texttt{getTy}(S_{\mathsf{stmt}}, suc)$	is delayed
6	:	$\texttt{!hasTy}(suc, Nat \to Nat)$	is added
7	:	$\texttt{!hasTy}(zero, Nat)$	is added
7	:	$\texttt{getTy}(S_{\mathsf{stmt}}, f), \texttt{getTy}(S_{\mathsf{stmt}}, suc), \texttt{getTy}(S_{\mathsf{stmt}}, zero)$	are resolved

Mutual references. An interesting case that arises when implementing forward references is when 2 statements mutually reference each other. Consider the following example:

```
postulate Nat : Set;
postulate zero : Nat;
postulate suc : Nat -> Nat;

def g : Nat -> Nat;
g = \n. f (suc n);

def f : Nat -> Nat;
f = \n. g n;
```

This code correctly type checks in our Statix implementation, because the mutual dependency isn't in the signatures of the definitions. Consider definition g first. We can easily check that $Nat \to Nat : Set$ and update S_{stmt} with the `!hasTy[g, Nat -> Nat]` binding. It is crucial to add this binding before checking the body of g, because it will get stuck on the reference to f. Similarly, we can check f's signature and make a binding for it before we check the body. When the bindings for the signatures are created, the queries for the type of g and f also become unstuck, and the delayed constraints will resume.

If the mutual dependency appears in the signatures themselves, then the checker cannot add either binding without first resolving the other. In that case, the constraints form a cyclic dependency at the type level. Such programs are not accepted by the current implementation.

4.6 Modules

This section describes another feature of our language that uses non-lexical scoping: modules. The concept of modules exists in many programming languages, but their implementations differ significantly. In some languages, modules primarily serve as compilation units or packages. In others, they are used to express abstraction boundaries, interfaces, or pa-

rameterized components. For this thesis, we are interested in a narrower aspect of modules, namely the interaction with name resolution.

4.6.1 Modules as name-resolution constructs

Modules can be viewed as constructs that organize declarations into named scopes. From this perspective, a module is not primarily a file, package, or compilation unit, but a scope that can be referred to by name. The declarations inside a module may be accessed through qualified names, or they may be made available for unqualified lookup by opening the module.

Lexical scoping alone is usually not sufficient to describe module lookup, because a declaration may become visible in a scope where it was not syntactically declared. For example, if a module is opened, names declared inside that module can be resolved from the opening scope. Similarly, a qualified name such as `A.B.x` requires lookup to proceed through a sequence of module scopes rather than through the usual lexical parent relation alone. The goal of this implementation is therefore not to model every aspect of module systems, but to capture the name-resolution behavior induced by modules. In particular, we focus on modules as named scopes, nested modules as hierarchical scopes, qualified names as path-based lookup, and open declarations as visibility links between scopes.

4.6.2 A core module fragment

Our Statix implementation of modules consists of declarations, nested modules, qualified access, and open statements. A module declaration introduces a named scope containing the statements in its body. For example, in the following program, the statement `x` belongs to the scope introduced by module `A`:

```
module A {
  def x : Set;
  x = ...
}
```

Modules may also be nested. A nested module introduces a new scope inside the scope of the surrounding module:

```
module A {
  def x : Set;
  x = ...

  module B {
    def y : Set;
    y = ...
  }
}
```

In this example, `B` is declared inside `A`. The declaration `y` belongs to the scope of `B`, while `x` belongs to the surrounding scope of `A`. Since `B` is nested inside `A`, declarations inside `B` may refer to declarations from `A` using the usual lexical lookup behavior.

The language also supports qualified access. A declaration inside a module can be referred to by prefixing it with the module path:

```
let z = A.B.y;
```

Resolving `A.B.y` requires resolving `A`, then resolving `B` inside the scope associated with `A`, and finally resolving `y` inside the scope associated with `B`. Thus, qualified names make the structure of module scopes explicit in the name being resolved.

Lastly, the language supports opening a module:

```
module C {
  open A;
  def z : Set;
  z = x;
}
```

The declaration `open A` makes the exported declarations of `A` available for unqualified lookup inside `C`. After opening `A`, the name `x` can be resolved in `C`. This is the main source of non-lexical scoping in the module implementation since `A` is not lexical parent of `C`.

4.6.3 Scope-graph representation

To support module declarations and openings, we have to adapt the existing scope graph edges and queries. Firstly, it is worth mentioning that declaring or opening a module are both statements in our language:

Listing 4.7: Module statements syntax

```
Stmt.Module = [module [ID] { [Stmt*] }]
Stmt.Open = [open [ID];]
```

Module declaration. Like definitions and postulates, declaring a module should create a binding in the top-level scope in which it was defined, such that we can refer to it by name. The module's body is type checked in a child scope of the top-level that we link via a P edge. This scope is important as it holds all the bindings created within the module which we may later import. Since a module is neither a type nor a substitution, we need a new relation `!modBind[ID, scope]` that associates the name of the module with the body's scope. With this setup, it is not possible to reference definitions from inside a module accidentally[5], because the module's body will be a sibling scope and not a direct parent. Primitives such as `getTy` or `lookupSubst` can reach the top level, but will not enter the module's scope.

Opening modules. Accessing a module is achieved by resolving its name. To do this we create a new primitive `getModule : (scope, ID) -> scope` that queries for a `modBind` relation containing `ID` by considering only paths of the form $P*$. Paths that contain fewer P edges are preferred since we want to resolve to the lexically closest module. If the query returns, then we obtained the reference to the module's scope. Opening this module will create a new I edge between the current scope and the module's scope. We choose to model imports with a different edge as they will allow us to distinguish between names that are resolved lexically and names that are resolved through importing.

```
module B {
  postulate x : Set;
}
module A {
  open B;
  x
}
```

[5]Without qualified access or opening the module

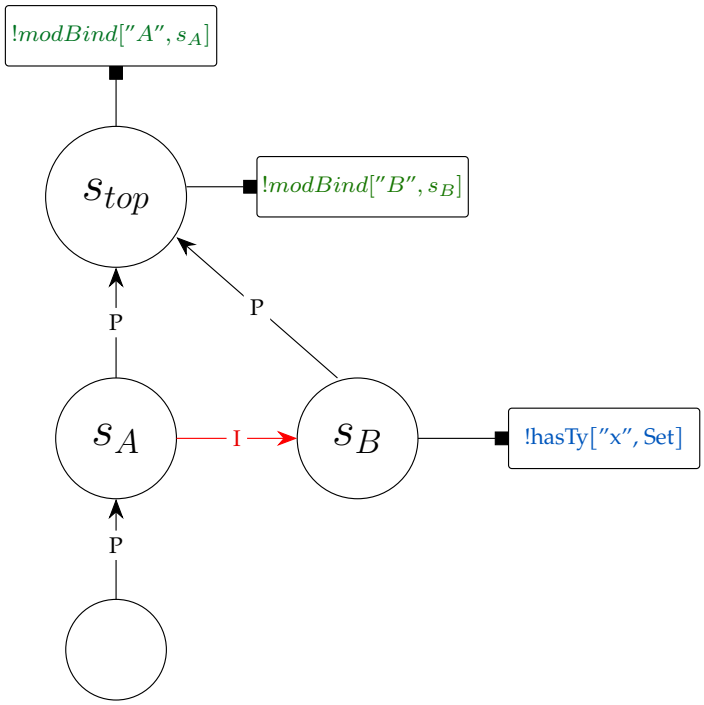


Figure 4.2: Resulting scope graph of 4.6.3

Name resolution. Lastly, because we have added a new type of edge, we need to adapt the current name resolution policy to be able to resolve names from opened modules. Instead of only considering paths of only P edges, `getTy` and `lookupSubst` should accept paths of the form $P * I?$. The last I edge should appear zero or one time, as we allow name resolution to walk only a single import.

A subtle aspect of this name resolution procedure is that we may find multiple declarations with the same name. Conflicting declarations may come from the local scope or from the opened modules. Programming language implementations take different approaches to this problem. For instance, in Agda you can open a module and choose to hide certain names. Normally, if multiple definitions with the same name are in scope it would cause an error, so hiding the names that you don't want to import is helpful. OCaml takes a different approach where local names are always preferred over imported ones. In Statix we will implement the latter as it fits better with scope graphs and the query policy. The name resolution primitives should prefer paths $P*$ over $P * I$, even if the path with I edges is shorter. Unfortunately, this cannot be modeled with Statix's prefix comparison of paths. Consider the paths PPI and PPP. We can say that we prefer the second, with the label order $P < I$ (Statix prefers the path with the smallest order). But in a case where we can only resolve the name through imports we might need to compare the paths: PPI and PI. With the same label order, Statix would choose the first path. This is not desirable, as we want to resolve a name by walking the closest import. Therefore, we need to execute name resolution in 2 phases. First we will query for the name by walking P edges only. If we cannot find the declaration, we will search again on paths $P * I$. Both queries will prefer the shortest paths.

Chapter 5

Inference and Unification

A practical type checker should aim to infer the type of as many expressions as possible. The bidirectional approach highlighted in Chapter 4, is able to infer the **type** of variables, let expressions and applications. However, implementations of dependently typed languages also offer the possibility of inferring **terms** that the user omits.

5.1 Typed holes

Typed holes are the mechanism for creating an unsolved variable that the checker knows to have a certain type. In our language, there is syntax that allows users to explicitly create holes:

```
Expr.Hole = "_"
```

Holes are expressions in the language, but are expected to be solved by the checker into a concrete term. This process is known in the literature as elaboration [19]. Elaboration in dependently typed languages is very complex and may be used for more than solving holes. For instance, in Agda the elaborator can insert holes in places where implicit arguments are expected and the user can omit them in the surface syntax. The elaboration component of our Statix type checker is represented by refinement of terms via the infer and check routines. Both check and infer will refine holes into **metavariables**. Because our language performs higher order unification, this refinement process is necessary to capture the dependencies of the hole. Dependencies are bound variables that are in scope when the hole is created, and are allowed to appear in the solution of the metavariable. Dependencies are captured by applying them to the metavariable in the resulting refined term. As an example consider the creation of the hole:

```
postulate A : Set;
postulate f : (x : A) -> (y : A) -> (x == y) -> A;

(\x. f x _ refl) :: A -> A
```

The expected type for the hole is `A`. Furthermore, the only bound variable that it can depend on is `x`. Then, the elaboration of this hole will produce the following internal representation:

```
postulate A : Set;
postulate f : (x : A) -> (y : A) -> (x == y) -> A;

(\x. f x (?m x) refl) :: A -> A
```

where `?m x :: A`. It is interesting to examine the type of the newly created metavariable $?m$. We observe that it should be a function type since it is applied on x. In this case $?m : (x :$

$A) \rightarrow A$. For the general case, a hole of type T will be refined to a metavariable $?m$ with dependencies $x_1 : A_1, x_2 : A_2, ...$ where $?m : (x_1 : A_1) \rightarrow (x_2 : A_2) \rightarrow ... \rightarrow T$. Note that identifiers x_1, x_2 may appear freely in T, since $?m$ type can be dependent.

The bidirectional type checking can easily support holes. If we are in check mode and the term is a hole, then the type is known and we can create the metavariable accordingly. If we encounter a hole in infer mode, we don't know which type should be used. We can create another hole of type Set to represent the unsolved type of the former hole. The hole for the type might be solved from later constraints, otherwise the checking of the program will fail.

5.2 Scoping of metavariables

Supporting metavariables reveals a unique challenge regarding scoping. A metavariable can be in any of the two states: solved or unsolved. This affects how beta reduction works in the language. Recall that weak head normal form is performed with the rule `whnf : scope * Expr -> (scope * Expr)`. In one scope s trying to reduce a metavariable would remain stuck since it is unsolved, while in another scope s' the metavariable should be reduced to its solution. We show the following example where this behavior occurs:

```
def f: (T : Set) -> (x : Nat) -> (P : (y : T) -> x==0) -> Vec T x
...

f Set _ (\n. refl)
```

We call s the scope in which the "_" is refined to a metavariable $?m$ and bound to x. Consider s' to be the scope in which `refl` is type checked against `x == 0`. s' is a descendant of s, although not a direct one. This scope generates the constraint that x should unify with 0 which gives the solution $?m = 0$. However, when the return type `Vec T x` is reduced, we are in a scope s_{ret} that is not a descendant of s'. s_{ret} will see the bindings for P, x or T, but it cannot see bindings created in s' (such as y). Bindings in Statix are immutable, so the original binding for x has the unsolved metavariable, and therefore when `Vec T x` is reduced we are not able to substitute 0 for x. It follows that a form of non-lexical scoping is required in order to accurately model metavariables. Metavariables should act as global objects, which once solved should be visible in all scopes.

5.3 Metavariables in scope graphs

In this section we will discuss how metavariables fit within the scope graph framework. We will also examine different approaches to model a binding that can change its value over time.

Bootstrapping Statix unification. When using scope graphs, the bindings that we create in a scope are immutable. However, an ideal approach for handling metavariables would create a binding for the unsolved state, which is later modified **in-place** when the solution becomes available. This similar behavior is present in Statix's own unification variables. In Statix, users can create unification variables with an unknown value. Statix's solver accumulates eventual equality constraints that involve this variable and can perform first-order unification to solve them. For instance, we can create the binding `!subst[?m, A]`, where `A` is an unknown Statix unification variable. Later constraints on `A` such as `A == Set`, will update the state of the variable to solved, with value `Set`. More importantly, the new state of `A` will also be visible in the binding that we created for the metavariable. This approach settles how we can solve a metavariable, and make the solution visible for later reductions. However, it does not solve the issue of knowing if a metavariable is unsolved and therefore should not be reduced.

Suppose that we want to replace $?m$ with A as part of a reduction. Statix will delay the result of the reduction because A's value might change. Even if A is later solved, at the moment we performed the reduction the correct result would have been to return the metavariable. Unfortunately, there isn't a way to interact with Statix's solver and query whether A is solved or not. Therefore, this solution isn't currently viable in Statix.

Modelling a global mutable scope. In section 5.2, we discussed that metavariables should live in a mutable global scope. This is not a native Statix feature, but can be simulated with scope graphs. With this approach we start by creating a completely different scope to the lexical scopes used for type checking. Mutation is implemented by extending the original scope with a child scope that contains the updated binding. The idea is similar to the append-only log [27] where updates are recorded sequentially in a file instead of modifying the structure in-place. This method can reveal the entire history of changes that happened on metavariables. The history is represented as a linear chain[1] of scopes and only the reference to the lowest descendant is needed to access all bindings. We will call the lowest scope in the chain s_{meta}.

In Statix, we will represent the 2 possible states of metavariables by 2 different relations:

1. `!hasTy[?m, T]` is re-used to represent that a metavariable `?m` of type T exists. If this is the only relation in which `?m` appears, it follows that the metavariable is not instantiated.

2. `!isInst[?m, V]` represents an instantiation of `?m` with expression `V`.

The chain of meta scopes will be connected by M edges. Figure 5.1 shows the resulting scope graph after creating a metavariable of type Set → Set, that is later instantiated with \x. Set:

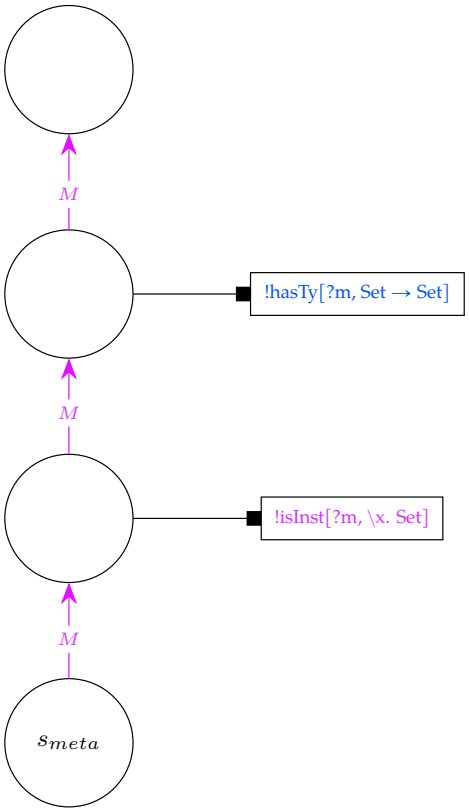


Figure 5.1: Example scope graph with metavariables

[1] Each scope has a single parent and at most one descendant

State pattern in a language without mutability. Both the check and infer routines will have to manage and mutate the metavariable scope. Statix is a functional constraint language that doesn't provide a way to store a mutable variable. Therefore, check and infer will have to receive s_{meta} as input and return the potentially modified chain s'_{meta}. This is a standard pattern in functional languages called state-monad pattern, where computations are modelled as functions from an initial state to a value paired with a final state [31]. Every rule that can create or instantiate metavariables will apply this pattern.

Interaction with name resolution Metavariables are also a form of identifiers. They are uniquely identified by the scope in which they were created, they have a type and optionally a value. The `Id` sort is extended with constructor `Meta : scope -> Id`, which receives the unique scope reference of the metavariable. Like all the other identifiers, the primitives we designed for name resolution should be able to interact with the metavariable scope. For instance, `getTy : scope * Id -> Expr` should be able to obtain the type of any identifier, including metavariables. We therefore extend `getTy` to take s_{meta} as an additional parameter, that will be used when the given identifier is a metavariable. We have also implemented a similar rule for obtaining the instantiation of a metavariable.

5.4 Higher order pattern unification in Statix

In the previous sections we have laid the necessary foundation for managing metavariables in the scope graph framework. In the following we will discuss how higher order pattern unification can be implemented using these building blocks.

Unification is a procedure that might be used when trying to establish that two terms are equal. If those terms contain metavariables, we will obtain different constraints on them. There are many places where equality is required in our type checker such as: the inferred type of a term matches its annotation, we test that two terms are definitionally equal, etc. We define equality as a Statix rule with the following signature:

```
equal : scope * (scope * Expr) * (scope * Expr) -> scope
equal (sMeta, (sA, A), (sB, B)) = ...
```

The `equal` rule takes 3 parameters: the reference to s_{meta}, closure of a term A and a closure of term B. Notice that the rule is expected to return a scope, namely a new reference to s_meta. If unification is performed, it will mutate the metavariable scope. Furthermore, if we discover that A and B are not equal, the rule will fail and type checking will not recover.

The first step that the equality rule will do is to reduce both terms up to weak head normal form. Depending on the head expressions of the two terms we can be in the three following cases discussed in Chapter 3:

- Rigid-Rigid
- Flex-Rigid
- Flex-Flex

It is also helpful to convert A and B to spine form representation after reducing to $whnf$. With this form terms have the form $hd\ [arg_1, arg_2, ...]$ instead of $App(App(App(hd, arg_1), arg_2), ...)$. Dealing with Rigid-Rigid problems is done by pattern matching on all the possible rigid heads in the language. A and B should have the same head but can have different arguments, meaning we need to recursively check for the equality of the arguments.

Rigid-Rigid decomposition rules

$$\frac{}{s_1, s_2 \vdash Set \equiv Set}\ \textsc{Set} \qquad \frac{}{s_1, s_2 \vdash refl \equiv refl}\ \textsc{refl}$$

$$\frac{s_1, s_2 \vdash A \equiv C, \quad B \equiv D}{s_1, s_2 \vdash (A == B) \equiv (C == D)}\ \textsc{Eq} \qquad \frac{s_1, s_2 \vdash x \equiv y}{s_1, s_2 \vdash Var(x) \equiv Var(y)}\ \textsc{Var}$$

$$\frac{\alpha \text{ fresh} \quad [x \mapsto \alpha]s_1, [y \mapsto \alpha]s_2 \vdash A \equiv B}{s_1, s_2 \vdash \backslash x.A \equiv \backslash y.B}\ \textsc{Lam}$$

$$\frac{\alpha \text{ fresh} \quad s_1, s_2 \vdash A \equiv C \quad [x \mapsto \alpha]s_1, [y \mapsto \alpha]s_2 \vdash B \equiv D}{s_1, s_2 \vdash (x : A) \to B \equiv (y : C) \to D}\ \textsc{Pi}$$

$$\frac{s_1, s_2 \vdash e_1 \equiv e_2}{s_1, s_2 \vdash \mathsf{Ann}(e_1, T) \equiv \mathsf{Ann}(e_2, T')}\ \textsc{Ann} \qquad \frac{s_1, s_2 \vdash e_1 \equiv e_2 \quad s_1, s_2 \vdash x_1 \equiv y_1, x_2 \equiv y_2, \ldots}{s_1, s_2 \vdash e_1\ \vec{x} \equiv e_2\ \vec{y}}\ \textsc{Stuck-terms}$$

It is important to mention that before pattern matching on the arguments of $\equiv$, the checker will reduce to weak head normal form. This makes sure that substitutions we saved are performed when needed.

Flex-Rigid problems require more careful handling. In general such constraints can either be solved, fail or be postponed. Our implementation cannot currently postpone constraints as it is not clear how to do so in Statix. However, we can recognize the cases where such a constraint can be successfully solved or should fail. Considering the problem to be in the form $?M\vec{x} \equiv R$, we perform the checks required by the higher order unification pattern algorithm:

1. **Occurs check**: We obtain the free variables of the rigid term R and check that $?M$ doesn't appear in them.
2. **Pattern check**: All arguments that $?M$ is applied on should be in the pattern fragment, namely they should all reduce to variables that are bound in this scope (bound variables).
3. **Linearity check**: After pattern check, we need to ensure that the bound variables are all unique.
4. **Scope check:** Compute the free variables $FV(R)$ and verify that they are contained in the set of variables visible from the scope in which $?M$ was created.

To implement these checks in Statix we require functions to reveal the free variables of a term and to obtain the binders that we are under in the current scope. Obtaining the free variables from term R is easy to do in our Statix representation. Free variables will have a `Fresh` constructor and will not be under a binder with the same name in R. Bound variables are obtained by querying for the `binder` relation starting from the current scope in which we perform the unification. This will return a list of all binders that we are under when unifying. Furthermore, all arguments that $?M$ is applied on should be elements of this list. If all the above conditions are met we can **invert** and obtain the solution for $?M$. The solution will have the form: $\backslash x_1.\ \backslash x_2.\ \backslash \ldots R$ and will be guaranteed to be well scoped.

Flex-Flex problems are hard to deal with in Statix. Without the ability to postpone problems we need an approximation to this scenario. The only problem we can solve with certainty is of the form $?M \equiv ?N$, where we unify 2 different metavariables with no arguments. The solution is to make one metavariable equal to the other. For the rest of flex-flex problems we will fail type checking and signal that this is a flex-flex case.

5.5 Discussion

The implementation presented in this chapter demonstrates that higher-order pattern unification can be integrated with a type checker that represents name binding using scope graphs. In particular, the scope graph provides the information required to determine which bound variables a metavariable may depend on. During flex-rigid unification, this information can be used to check that the metavariable application belongs to the pattern fragment and to construct a well-scoped lambda abstraction as its solution. Therefore, the first-order representation of terms used by Statix does not prevent higher-order unification, provided that the binding context required by the unification algorithm can be recovered explicitly.

An important consequence of adding metavariables is that their scoping behaviour differs from that of ordinary program variables. A metavariable may be introduced in one lexical scope, solved while checking a term in another scope, and subsequently inspected from yet another part of the program. Its solution must therefore be globally visible, while the variables on which the solution is allowed to depend are still determined by the context in which the metavariable was created. The separate metavariable scope introduced in Section 5.3 reflects this distinction. It provides global access to the current state of a metavariable while retaining the lexical information necessary to restrict its possible solutions.

The implementation also exposes a limitation of the current integration. Higher-order pattern unification is most useful in an elaborator that can postpone constraints which cannot yet be solved and reconsider them after more metavariables have been instantiated. The current Statix implementation does not provide such a mechanism for the higher-order constraints introduced in this work. As a result, flex-rigid constraints can be solved when they already satisfy the pattern conditions, whereas constraints that might only become patterns later must be rejected. Support for flex-flex constraints is similarly restricted. The result of this chapter is therefore an integration of the higher-order pattern fragment with scope graphs, and not a complete implementation of dynamic higher-order unification.

Overall, these results give a positive answer to RQ1. Scope graphs can provide the binding information required by higher-order pattern unification, while metavariable state can be represented alongside the ordinary scope graph. At the same time, the implementation shows that binding representation is only one part of a practical elaborator. Richer support for constraint postponement and reactivation would be required to cover the broader class of unification problems encountered by dependently typed languages. The following chapter examines how these responsibilities change when binding and higher-order unification are instead provided directly by a higher-order meta-language.

Chapter 6

A comparison of scope graphs and HOAS

In the preceding chapters, we presented the implementation of our dependently typed language in Statix, which represents binding structure explicitly using scope graphs. This representation is particularly useful for language constructs whose visibility cannot be described solely by the lexical nesting of the abstract syntax tree, such as modules and module openings. However, the explicit representation of binding also introduces implementation work. Operations such as substitution, α-renaming, and higher-order unification must operate over a first-order representation of terms while respecting the binding information recorded in the scope graph. This raises the question of how the implementation would differ in a language in which binders and metavariables are primitive concepts.

To investigate this question, we implement the core of the same language in Elpi. Elpi is a dialect of λProlog designed for programs that manipulate syntax containing binders and unification variables [8]. It supports higher-order abstract syntax (HOAS), in which binders in the object language are represented by binders in the meta-language [24]. Consequently, operations such as substitution and α-renaming can be delegated to the Elpi runtime.

The purpose of this chapter is not to claim that either representation is universally preferable. Instead, we use the two implementations to identify their respective strengths. HOAS is particularly convenient for the locally scoped binders found in lambda abstractions and dependent function types. Scope graphs, by contrast, provide an explicit approach to name resolution, which is beneficial for modules, imports, qualified references, and other forms of non-lexical scope.

6.1 Logic programming in Elpi

An Elpi program consists of relations defined by logical clauses. A predicate declaration specifies the arguments of a relation and may additionally specify the intended input and output modes of those arguments. For example, the main judgments of our type checker are declared as follows:

Listing 6.1: The principal relations of the Elpi type checker

```
pred infer i:tm, o:tm, o:tm.
pred check i:tm, i:tm, o:tm.
pred whnf i:tm, o:tm.
pred conv i:tm, i:tm.
```

Similarly to our Statix approach, the judgment `infer T T' A` states that the source term `T` elaborates to `T'` and synthesises type `A`. Moreover, `check T A T'` states that `T` checks against

the expected type `A` and elaborates to `T'`. The separation between source and elaborated terms allows occurrences of `hole` in the input to be replaced by Elpi unification variables.

Variables in a clause are implicitly universally quantified. During proof search, Elpi selects a clause whose conclusion unifies with the current goal and recursively attempts to prove the premises of that clause. Unification therefore serves both as a mechanism for pattern matching and as a mechanism for constructing outputs. For instance, the rule for inferring the type of an annotation can be expressed as follows:

Listing 6.2: Type inference for annotations

```
infer (ann T A) (ann T1 A1) A1 :-
  check A set A1,
  check T A1 T1.
```

The conclusion is first unified with the current inference goal. The first premise checks that the annotation denotes a type, while the second checks the annotated term against that type. Variables such as `T1` and `A1` are instantiated as the recursive calls elaborate the corresponding subterms.

Generic and existential variables. Elpi distinguishes between eigenvariables, introduced by the `pi` quantifier, and unification variables, which are existential unknowns. Operationally, a goal of the form

```
pi x\ Goal
```

introduces a fresh rigid constant for `x` while solving `Goal`. This fresh constant cannot later be unified with an arbitrary term. It is therefore suitable for representing an object-language variable while traversing a binder.

The `sigma` quantifier instead introduces a fresh local unification variable:

```
sigma M\ Goal
```

The position of these quantifiers determines the variables that may occur in a solution. In particular, `pi x\ sigma M\ M = x` has a solution because `M` is introduced while `x` is in scope. In contrast, `sigma M\ pi x\ M = x` fails because an existential variable introduced before `x` may not later capture it. However, an outer metavariable may depend on a new binder when it is applied to it explicitly:

```
sigma M\ pi x\ M x = x
```

In this case, Elpi can solve `M` with the function `x\ x`. This restriction is essential for our treatment of holes. A metavariable occurring below the binders `A` and `x` is represented by an application such as `M A x`. Its eventual solution may depend on either binder, but cannot refer to a variable that was not in scope when the metavariable was created.

Hypothetical clauses. The typing context is represented using hypothetical clauses rather than an explicit list. We introduce the predicate

```
pred localTy i:tm, o:tm.
```

where `localTy X A` records that `X` has type `A`. A variable is typed by retrieving the corresponding hypothetical clause:

Listing 6.3: Retrieving the type of a local variable

```
infer X X A :-
  localTy X A.
```

When the checker moves below a binder, it extends the current program temporarily using the `==>` connective:

```
pi x\ localTy x A ==> Goal
```

The hypothetical fact `localTy x A` is available while `Goal` is being solved and is removed afterward. The combination of `pi` and hypothetical clauses therefore corresponds directly to extending a typing context with a fresh assumption:

$$\Gamma, x : A \vdash t : B.$$

The context is managed by the Elpi runtime and follows the lexical structure of the recursive computation, therefore it is not necessary to pass a context parameter explicitly through every typing rule.

6.2 Defining the language in HOAS

Listing 6.4 presents the Elpi representation of the core language considered in this chapter.

Listing 6.4: The syntax of the language in Elpi

```
kind tm type.

type set  tm.
type eq   tm -> tm -> tm.
type refl tm -> tm.

type app  tm -> tm -> tm.
type lam  (tm -> tm) -> tm.
type arr  tm -> (tm -> tm) -> tm.

type ann  tm -> tm -> tm.
type hole tm.
```

The constructor `set` represents the universe of types. For this language, we use the simplified rule `set : set` and omit the universe hierarchy. The constructor `app F X` represents object-language application. The constructors `lam` and `arr` contain meta-level functions and are therefore the two constructors that employ HOAS. An object-language abstraction $\lambda x.t$ is represented as:

```
lam (x\ T)
```

while a dependent function type $(x : A) \to B(x)$ is represented as:

```
arr A (x\ B)
```

There is no separate constructor for bound variables. Occurrences of `x` in `T` and `B` are Elpi variables bound by the surrounding meta-level abstraction. As a result, object-level α-equivalence is obtained through meta-level α-equivalence.

The remaining constructors are first order. The term `ann T A` associates the term `T` with the type annotation `A`. The term `eq L R` denotes the equality type between `L` and `R`. The constructor `refl` is checked against an equality whose two arguments are definitionally equal. Lastly, `hole` represents the syntax for a placeholder that is elaborated to a unification variable. Finally, we can write the polymorphic identity function from our object language in Elpi:

Listing 6.5: The polymorphic identity function in HOAS

```
ann
  (lam (A\ lam (x\ x)))
  (arr set (A\ arr A (x\ A)))
```

This term is equivalent to:

$$\lambda A.\lambda x.x :: (A : \mathsf{Set}) \to (x : A) \to A.$$

6.2.1 Bidirectional type checking

We implement the language using bidirectional typing. Terms such as lambda abstractions and reflexivity are checked against an expected type, whereas applications and explicitly annotated terms are inferrable. The rule for the universe is immediate:

```
infer set set set.
```

Dependent function types require the domain to be a type and the codomain to be a type in a context extended with a variable of the domain type:

Listing 6.6: Inference for dependent function types

```
infer (arr A B) (arr A1 B1) set :-
  check A set A1,
  pi x\ localTy x A1 ==> check (B x) set (B1 x).
```

The use of `B x` opens the object-language binder by applying the meta-level function `B` to the fresh eigenvariable `x`. The elaborated codomain is similarly returned as `B1 x`. Since `x` is passed explicitly to `B1`, the elaborated codomain may depend on the bound variable.

The rule for lambda abstractions follows the same pattern:

Listing 6.7: Checking a lambda abstraction

```
check (lam F) Expected (lam G) :-
  whnf Expected (arr A B),
  pi x\ localTy x A ==> check (F x) (B x) (G x).
```

The expected type is first reduced to a dependent function type. The fresh variable `x` is then assigned the domain type `A` using a hypothetical clause. The body `F x` is checked against the instantiated codomain `B x`. This rule closely follows the declarative typing rule

$$\frac{\Gamma, x : A \vdash t : B(x)}{\Gamma \vdash \lambda x.t : (x : A) \to B(x)}.$$

The Elpi representation does not require an explicit freshness side condition. The eigenvariable generated by `pi` is fresh by construction, and its scope is limited to the recursive call.

Application infers a dependent function type for the function position, checks the argument against its domain, and instantiates the codomain with the elaborated argument:

Listing 6.8: Inference for dependent application

```
infer (app F X) (app F1 X1) Result :-
  infer F F1 FunctionType,
  whnf FunctionType (arr A B),
  check X A X1,
  Result = B X1.
```

The assignment `Result = B X1` performs substitution at the type level. In the scope graph implementation, obtaining the result type requires an explicit operation that substitutes the argument for the bound variable in the codomain. In the HOAS representation, the same operation is easier to perform via meta-level function application.

6.2.2 Substitution and normalisation

The principal advantage of HOAS becomes particularly clear in the implementation of β-reduction. In our language, we still require a predicate for converting terms to weak head normal form since the `app` constructor allows for terms with β-redexes. The weak-head reduction of a lambda application can be written as follows:

Listing 6.9: Beta reduction through meta-level application

```
whnf (app F X) R :-
  whnf F F1,
  whnf-app F1 X R.

whnf T T.

whnf-app (lam Body) X R :-
  whnf (Body X) R.

whnf-app F X (app F X).
```

In the `whnf-app` rule, `Body X` is the result of substituting `X` for the variable bound by `Body`. The Elpi runtime performs this substitution using meta-level β-reduction, which is consequently capture avoiding without requiring an explicit traversal of the object-language term (e.g. to freshen bound variables). For example, `app (lam (x\ x))set` reduces by evaluating the meta-level application `(x\ x)set` whose result is `set`. A body containing nested binders is handled in the same way. The runtime is responsible for renaming meta-level binders when necessary to avoid capture.

By contrast, the scope graph representation keeps binders and references as first-order syntax. Scope graphs determine which declaration a variable occurrence denotes, but they do not themselves perform substitution. Our Statix implementation must therefore define substitution explicitly and preserve the binding relationships of the substituted term. This includes avoiding name capture when a substitution moves a term below a binder.

6.2.3 Equality and reflexivity

The equality constructor is typed by first inferring the type of its left argument and then checking the right one against the same type:

Listing 6.10: Inference for equality types

```
infer (eq L R) (eq L1 R1) set :-
  infer L L1 A,
  check R A R1.
```

The constructor `refl` has a single argument x, and the resulting type should be $x \equiv x$:

Listing 6.11: Inferring reflexivity

```
infer (refl T) (refl T1) (eq T1 T1) :-
  infer T T1 _.
```

Combined with unification and normalization, reflexivity can establish equalities whose two arguments are not initially syntactically identical. A reflexivity proof has a type of the form $x \equiv x$. When this type is unified with an expected equality $z \equiv y$, both z and y must unify with the same term x, possibly after normalization. Consequently, reflexivity succeeds whenever the two arguments of the equality are definitionally equal. For example, the following term proves that applying the identity function to a value returns that value:

Listing 6.12: A reflexivity proof relying on beta reduction

```
ann
  (lam (A\ lam (x\ refl x)))
  (arr set (A\ arr A (x\
    eq
      (app (ann (lam (y\ y)) (arr A (y\ A)))
      x)
    x)))
```

The expected type of `refl x` is annotated as:

$$(\lambda y.y)\ x \equiv x.$$

The arguments are not structurally identical. The conversion relation reduces the left-hand side to `x`, after which the equality is accepted.

6.2.4 Holes and scoped metavariables

A syntax level occurrence of `hole` is elaborated to an Elpi unification variable. The important property is that the unification variable records the binders (eigenvariables) that were in scope at the point where the hole occurred. Take for instance the term

```
lam (A\ lam (x\ hole))
```

checked against the type

```
arr set (A\ arr A (x\ A))
```

The hole will be elaborated to a metavariable $?M$ that occurs as `M A x` inside the body. The arguments express the local context available to its solution. The metavariable may be solved with `x`, with a term depending only on `A`, or with a closed term, provided that the solution has the expected type. Inside Elpi, the context of a metavariable can be obtained through built-in functions. The `names L` function returns a list of all eigenvariables in scope, while `prune V L` unifies `V` with a unification variable that only sees the names in list `L`. Using these 2 primitives we can construct metavariables from the object language via Elpi's notation. Furthermore, Elpi performs a lot of work for free, regarding the management of metavariables. The higher-order unification algorithm comes with the necessary pattern checks and with the ability to delay constraints that are outside the pattern fragment. This is a considerable advantage to implementing the unification procedure manually.

6.3 HOAS as an alternative to scope graphs

The HOAS implementation demonstrates that a substantial part of the complexity of manipulating binders can be delegated to the meta-language. Nevertheless, it also exposes a limitation: the context managed by `pi` and hypothetical clauses is fundamentally lexical and stack structured. A hypothetical declaration is introduced while solving a subgoal and is removed when that subgoal finishes. This behavior matches lambda abstractions and dependent function types closely. A variable introduced by a lambda is visible just in the body

of that lambda. However, not every name-binding construct follows this structure. Module systems introduce visibility relations between parts of the program that are not necessarily related in the AST. Opening a module, resolving a qualified name, and forward references require relationships that don't follow the tree structure of the program. While it is possible to encode such relationships in Elpi, for example by passing an explicit environment or by maintaining relations describing modules and their exported declarations, once this is done, the implementation no longer obtains the main simplicity of the hypothetical context. Name resolution becomes a separate user-defined mechanism layered on top of the HOAS representation. Consider the program:

Listing 6.13: A non-lexical visibility relation introduced by opening a module

```
module M {
def A : Set;
A = Set;
}

module N {
open M;
def B : Set;
B = A;
}
```

The declaration of `A` is not a lexical ancestor of its use in module `N`. In the scope graph representation, resolving `A` follows an import or opening edge from the scope of `N` to the body scope of `M`. The relationship is represented directly in the graph and can be reused by every reference appearing in `N`. In a purely HOAS based implementation, the same effect cannot be obtained by opening an object-language binder. The module declaration and its use occur in different branches of the syntax tree, and the visibility of `A` is created by the `open` statement rather than by lexical nesting. The implementation must therefore locate the module, retrieve its exported declarations, and make them available while checking the remainder of the module body. Shadowing and qualified access require additional policies around this environment.

Chapter 7

Related work

This thesis lies at the intersection of dependent type elaboration, higher order unification, and declarative name binding. Each of these topics has been studied independently. Higher order unification is commonly used during the elaboration of dependent types, while scope graphs and Statix provide a declarative approach to name resolution and type system specification. Higher Order Abstract Syntax provides another way to represent languages with binders. This chapter places the approach developed in this thesis in relation to these areas.

7.1 Higher order unification and dependent type elaboration

Higher order unification has a long history in logic programming and type checking. Miller identified the higher order pattern fragment, which restricts the arguments of unknown functions to distinct bound variables [17]. Within this fragment, unification remains decidable and solutions have a most general form. The pattern fragment is therefore useful for systems that need to solve equations involving terms under binders.

Dependent type checking gives rise to such equations during inference and elaboration. Mazzoli and Abel study the relation between type checking and unification and show how type checking can be formulated around the generation and solution of unification constraints [16]. Work on elaboration for dependent type theory also uses unification to reconstruct information that is omitted from source programs [19]. Similar problems occur in practical dependent type systems such as Agda [23].

Restricting all generated constraints to the pattern fragment is not always possible. Abel and Pientka describe dynamic pattern unification, where a constraint that cannot currently be solved may be delayed until more information becomes available [2]. Their work also describes pruning, which can remove dependencies that prevent a constraint from satisfying the pattern restrictions. These techniques provide a more complete treatment of higher order constraints than the implementation in this thesis. The Statix implementation presented here handles constraints that can already be recognized as solvable pattern problems. It does not currently provide the postponement mechanism required for dynamic pattern unification.

The aim of this thesis is therefore not to introduce a new general higher order unification procedure. Instead, it studies how the information required by higher order pattern unification can be obtained when name binding is represented through scope graphs. In particular, the implementation uses the scope structure to determine which bound variables may occur in a solution. It also introduces an explicit representation for the state associated with unknown terms during inference.

7.2 Scope graphs and dependent typing in Statix

Scope graphs were introduced as a general representation of name resolution [21]. A scope graph represents scopes and the visibility relations between them explicitly. This allows the same name resolution framework to describe both lexical binding and relations introduced by constructs such as modules and imports. Statix builds on scope graphs to describe name resolution together with typing constraints [4, 29].

Brouwer previously investigated the implementation of a dependently typed language in Statix [14]. That work shows how core features of dependent typing can be expressed in the Statix setting. However, it does not include higher order unification. The present work continues from this setting and focuses on the inference problems that require higher order solutions. The main additional problem is that a solution may depend on variables introduced by surrounding binders. The implementation therefore needs to relate the context of a unification problem to the binding information represented in the scope graph.

This also creates a difference from most presentations of higher order unification. In those presentations, the context of an unknown term is usually represented directly as part of the unification problem. For example, contextual representations can associate an open term with the variables on which it may depend [20]. In Statix, the corresponding information is already present in the scope graph. This thesis investigates how that representation can be reused during higher order pattern unification instead of introducing a separate representation of lexical binding.

7.3 Higher Order Abstract Syntax and logic programming

Higher Order Abstract Syntax provides a different approach to the representation of binders. In HOAS, binders in the object language are represented using binders in the meta language [24]. As a consequence, operations involving lexical binding can be delegated to the underlying language rather than implemented explicitly.

Logic programming languages based on lambda tree syntax make this approach executable. Elpi is an implementation of this style of logic programming and provides support for terms containing binders and for higher order pattern unification [8]. This makes it possible to express typing rules for lambda abstractions and dependent function types using the binding and context mechanisms of the meta language.

The Elpi implementation in Chapter 6 is used as a point of comparison for the Statix implementation. The comparison concerns the representation of binding rather than the higher order unification algorithm itself. Elpi provides direct support for lexical binders, substitution, and higher order unification. Statix represents binding explicitly and requires more of these operations to be implemented by the language specification. At the same time, scope graphs directly represent visibility relations that do not follow lexical nesting, such as module openings and qualified references.

The two approaches therefore emphasize different aspects of language implementation. The work in this thesis studies higher order inference in a setting where the explicit name resolution structure provided by scope graphs is retained, rather than replacing that structure with a higher order representation of all binders.

Chapter 8

Conclusion

This thesis investigated whether higher-order pattern unification can be combined with scope-graph-based name binding in the implementation of a dependently typed language. To study this question, we implemented a small dependently typed language in Statix and extended its type checker with higher-order inference. We subsequently implemented the core of the same language in Elpi in order to compare the scope-graph representation with Higher-Order Abstract Syntax.

With respect to RQ1, the Statix implementation shows that scope graphs can provide the binding information required by higher-order pattern unification. The variables available to a solution can be obtained from the scope graph and used to construct well-scoped higher-order solutions. Metavariables additionally require state that is visible independently of the lexical scope in which a solution is discovered, which led to the separate metavariable scope introduced in Chapter 5. The resulting implementation supports the higher-order pattern cases considered in this thesis, although the lack of constraint postponement restricts the set of unification problems that can be handled.

With respect to RQ2, the comparison with Elpi shows that the two approaches place complexity in different parts of the language implementation. HOAS delegates lexical binding, capture-avoiding substitution, and higher-order unification to the meta-language. Scope graphs require these operations to be made more explicit, but provide very good support for non-lexical name resolution, including modules, qualified references, and forward references.

Overall, this work shows that higher-order pattern unification is compatible with a scope-graph-based representation of name binding. The main remaining limitations arise not from scope graphs themselves, but from the additional solver behavior required by a more complete dependent type elaborator.

8.1 Future Work

There are several directions in which the implementation presented in this thesis could be extended.

Constraint postponement and reactivation. The most immediate limitation of the current higher-order unification procedure is the absence of constraint postponement. A constraint that is outside the pattern fragment cannot necessarily be classified permanently as unsolvable since later instantiations may simplify it into a pattern problem. Supporting this behavior would require a mechanism for suspending higher-order constraints and reactivating them when one of the metavariables on which they depend is instantiated. Such a mechanism would considerably reduce the number of constraints that the current implementation has to reject eagerly.

Pruning and more complete higher-order pattern unification. Another possible extension is pruning. Sometimes a unification problem cannot be solved because the proposed solution for a metavariable would refer to a variable that was not in scope when that metavariable was introduced. However, if the second metavariable does not actually need to depend on the variable, its context can be reduced so that the offending dependency disappears. The original unification problem may then become solvable. Implementing this pruning step would allow the current solver to handle more higher-order constraints.

Closer integration with the Statix solver. The current implementation represents metavariable state explicitly through a separate chain of scopes that is threaded through the type-checking rules. This design was necessary because ordinary scope-graph bindings are immutable and because the state of Statix unification variables cannot be inspected in the way required by the implementation. An interesting direction would be to investigate whether higher-order metavariables could be supported more directly by the Statix solver. In particular, allowing a Statix unification variable to "see" a local context or exposing mechanisms for observing the state of unification variables could remove part of the very explicit state management currently required.

8.2 Reproducibility

The implementation artifacts developed as part of this thesis consist of an Eclipse project containing the Statix specification of the dependently typed language, together with a set of Elpi source files used for the alternative HOAS-based implementation discussed in Chapter 6.

The Statix artifact contains the language definition, type-checking rules, scope-graph-based name resolution, metavariable handling, and higher-order pattern unification described in Chapters 4 and 5. The project can be opened in the Spoofax/Eclipse development environment and includes the examples and tests used during development to validate the implementation. The Elpi artifact contains the corresponding core language implementation used for the comparison with Higher-Order Abstract Syntax.

8.3 AI usage

Generative AI tools were used during the preparation of this thesis, limited to tasks such as improving my own wording, formatting figures, LaTeX code and researching documentation or new papers to read. They were not used as an authoritative source of scientific information. AI was not used to develop the theory behind the metavariable scoping in Statix, or the Statix code. For the ELPI implementation of the language, it was used to find built-in functions that can apply multiple arguments to an ELPI metavariable. In general, technical claims, definitions, algorithms, and comparisons included in this thesis were verified against the cited literature.

Bibliography

[1] Andreas Abel, Klaus Aehlig, and Peter Dybjer. "Normalization by Evaluation for Martin-Löf Type Theory with One Universe". In: *Electronic Notes in Theoretical Computer Science* 173 (2007), pp. 17–39. DOI: 10.1016/j.entcs.2007.02.025.

[2] Andreas Abel and Brigitte Pientka. "Higher-Order Dynamic Pattern Unification for Dependent Types and Records". In: *Typed Lambda Calculi and Applications*. Vol. 6690. Lecture Notes in Computer Science. Springer, 2011, pp. 10–26. DOI: 10.1007/978-3-642-21691-6_5.

[3] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.

[4] Hendrik van Antwerpen et al. "Scopes as types". In: *Proceedings of the ACM on Programming Languages* 2 (OOPSLA Oct. 24, 2018), 114:1–114:30. DOI: 10.1145/3276484.

[5] Adam Chlipala. "Parametric higher-order abstract syntax for mechanized semantics". In: *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming*. ICFP '08. Victoria, BC, Canada: Association for Computing Machinery, 2008, pp. 143–156. ISBN: 9781595939197. DOI: 10.1145/1411204.1411226.

[6] Miller Dale. "Mechanized metatheory revisited". In: *Journal of Automated Reasoning* 3 (Oct. 2019), pp. 625–665. DOI: 10.1007/s10817-018-9483-3.

[7] Luís Damas and Robin Milner. "Principal Type-Schemes for Functional Programs". In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '82. ACM, 1982, pp. 207–212. DOI: 10.1145/582153.582176.

[8] Cvetan Dunchev et al. "ELPI: fast, Embeddable, λProlog Interpreter". In: *LNCS*. Suva, Fiji, Nov. 2015. DOI: 10.1007/978-3-662-48899-7_32.

[9] Jana Dunfield and Neel Krishnaswami. "Bidirectional Typing". In: *ACM Computing Surveys* 54.5 (2022), 98:1–98:38. DOI: 10.1145/3450952.

[10] Jana Dunfield and Neelakantan R. Krishnaswami. "Complete and Easy Bidirectional Typechecking for Higher-Rank Polymorphism". In: *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*. ICFP '13. ACM, 2013, pp. 429–442. DOI: 10.1145/2500365.2500582.

[11] Sebastian Erdweg et al. "Evaluating and comparing language workbenches: Existing results and benchmarks for the future". In: *Computer Languages, Systems & Structures*. Special issue on the 6th and 7th International Conference on Software Language Engineering (SLE 2013 and SLE 2014) 44 (Dec. 1, 2015), pp. 24–47. DOI: 10.1016/j.cl.2015.08.007.

[12] Adam Gundry, Conor McBride, and James McKinna. "Type Inference in Context". In: *Proceedings of the Third ACM SIGPLAN Workshop on Mathematically Structured Functional Programming*. MSFP '10. ACM, 2010, pp. 43–54. DOI: 10.1145/1863597.1863608.

[13] Antonius J. C. Hurkens. "A Simplification of Girard's Paradox". In: *Typed Lambda Calculi and Applications*. Springer, 1995, pp. 266–278. DOI: 10.1007/BFb0014058.

[14] Brouwer Jonathan. "Dependently Typed Languages in Statix". In: (2023). URL: https://repository.tudelft.nl/record/uuid:7bf3c0f5-71fb-4e08-bcdb-1c873c7e1e63.

[15] Per Martin-Löf. *Intuitionistic Type Theory*. Studies in Proof Theory. Notes by Giovanni Sambin of a series of lectures given in Padua, June 1980. Naples: Bibliopolis, 1984.

[16] Francesco Mazzoli and Andreas Abel. "Type Checking through Unification". In: *CoRR* abs/1609.09709 (2016). arXiv: 1609.09709 [cs.PL].

[17] Dale Miller. "A Logic Programming Language with Lambda-Abstractions, Function Variables, and Simple Unification". In: *Journal of Logic and Computation* 1.4 (1991), pp. 497–536. DOI: 10.1093/logcom/1.4.497.

[18] Robin Milner. "A Theory of Type Polymorphism in Programming". In: *Journal of Computer and System Sciences* 17.3 (1978), pp. 348–375. DOI: 10.1016/0022-0000(78)90014-4.

[19] Leonardo de Moura et al. "Elaboration in Dependent Type Theory". In: *Automated Deduction – CADE-25*. Springer, 2015, pp. 14–29.

[20] Aleksandar Nanevski, Frank Pfenning, and Brigitte Pientka. "Contextual Modal Type Theory". In: *ACM Transactions on Computational Logic* 9.3 (2008), pp. 1–49. DOI: 10.1145/1352582.1352591.

[21] Pierre Neron et al. "A Theory of Name Resolution". In: *Programming Languages and Systems (ESOP 2015)*. Vol. 9032. Lecture Notes in Computer Science. Springer, 2015, pp. 205–231. DOI: 10.1007/978-3-662-46669-8_9.

[22] Ulf Norell. "Dependently Typed Programming in Agda". In: *Advanced Functional Programming*. Vol. 5832. Lecture Notes in Computer Science. Springer, 2009, pp. 230–266. DOI: 10.1007/978-3-642-04652-0_5.

[23] Ulf Norell. "Towards a Practical Programming Language Based on Dependent Type Theory". PhD thesis. Chalmers University of Technology and Göteborg University, 2007. ISBN: 978-91-7291-996-9.

[24] F. Pfenning and C. Elliott. "Higher-order abstract syntax". In: *SIGPLAN Not.* 23.7 (June 1988), pp. 199–208. DOI: 10.1145/960116.54010.

[25] Gordon D. Plotkin. "Call-by-Name, Call-by-Value and the λ-Calculus". In: *Theoretical Computer Science* 1.2 (1975), pp. 125–159. DOI: 10.1016/0304-3975(75)90017-1.

[26] Giselle Reis. *Curry-Howard Correspondence*. URL: https://web2.qatar.cmu.edu/cs/15317/lectures/04-curryhoward.pdf.

[27] Mendel Rosenblum and John K. Ousterhout. "The design and implementation of a log-structured file system". In: *ACM Trans. Comput. Syst.* 10.1 (Feb. 1992), pp. 26–52. DOI: 10.1145/146941.146943.

[28] Michael L. Scott. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann, 2015. ISBN: 978-0124104099.

[29] H. van Antwerpen. "Declarative Name Binding for Type System Specifications". English. Dissertation (TU Delft). Delft University of Technology, 2024. ISBN: 978-94-6384-704-9. DOI: 10.4233/uuid:4bf44aa1-779c-4a96-8c55-5e1b54e16119.

[30] Guido H. Wachsmuth, Gabriël D.P. Konat, and Eelco Visser. "Language Design with the Spoofax Language Workbench". In: *IEEE Software* 31.5 (2014), pp. 35–43. DOI: 10.1109/MS.2014.100.

[31] Philip Wadler. “Monads for Functional Programming”. In: *Advanced Functional Programming*. Springer, 1995, pp. 24–52. DOI: 10.1007/3-540-59451-5_2.

Acronyms

RQ research question

AI artificial intelligence

AST abstract syntax tree

HOU higher-order unification

HOAS higher-order abstract syntax

PHOAS parametric higher order abstract syntax

WHNF weak head normal form

NbE normalization by evaluation

IDE integrated development environment